

UVA DS2003: Communicating with Data

Fall 2026

Visualization Design:

Visual Encodings and Scale

Sarah Groves

Adapted from Hannah Bako

In class quiz – 10 minutes

What you will learn today

- Mark, Visual Channel, and Visual Encoding
- Chart Types: a Deconstructionist Approach
- Scale components

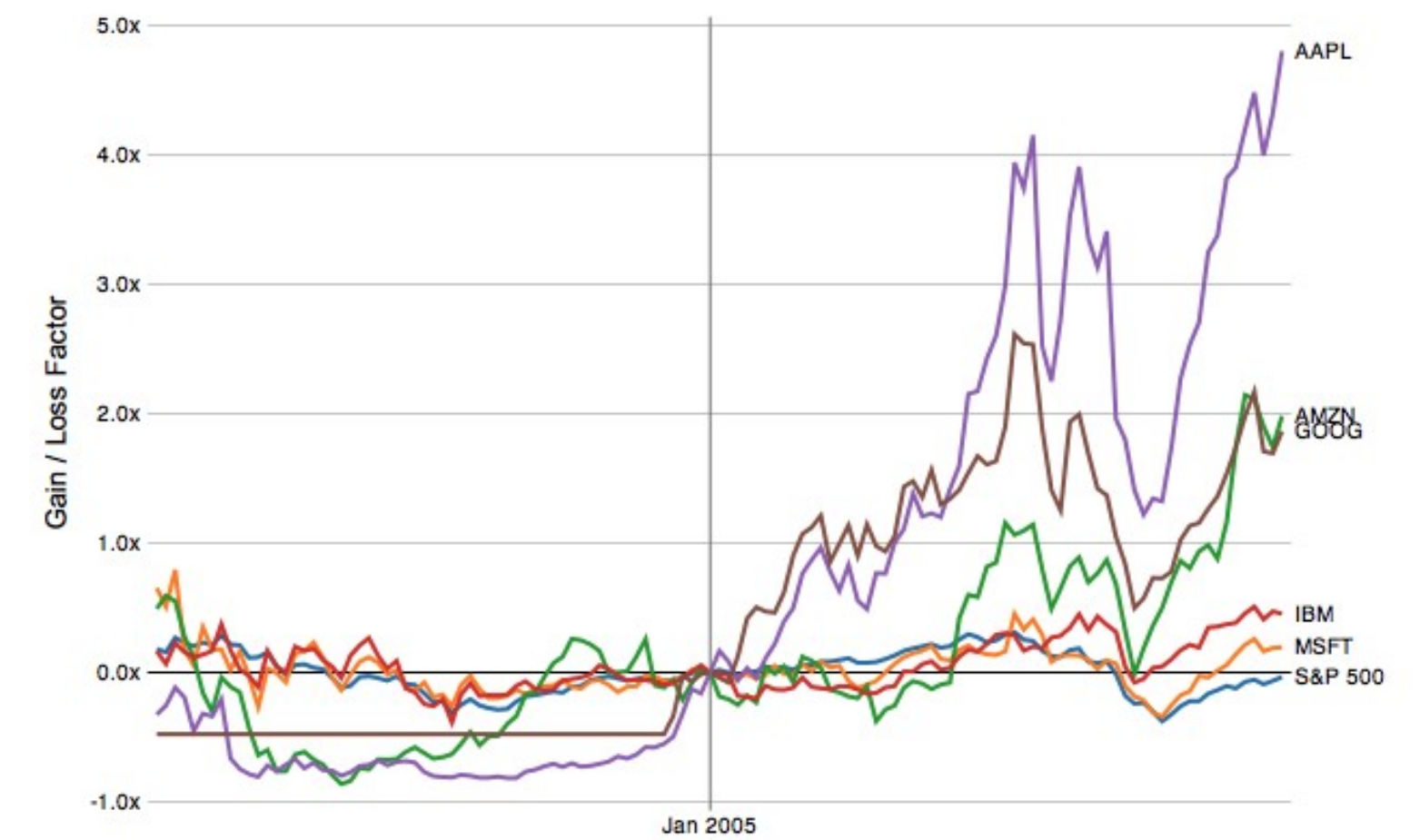
Reading

A Tour through the Visualization Zoo by Jeffrey Heer, Michael Bostock, and Vadim Ogievetsky

Overview of various visualization types

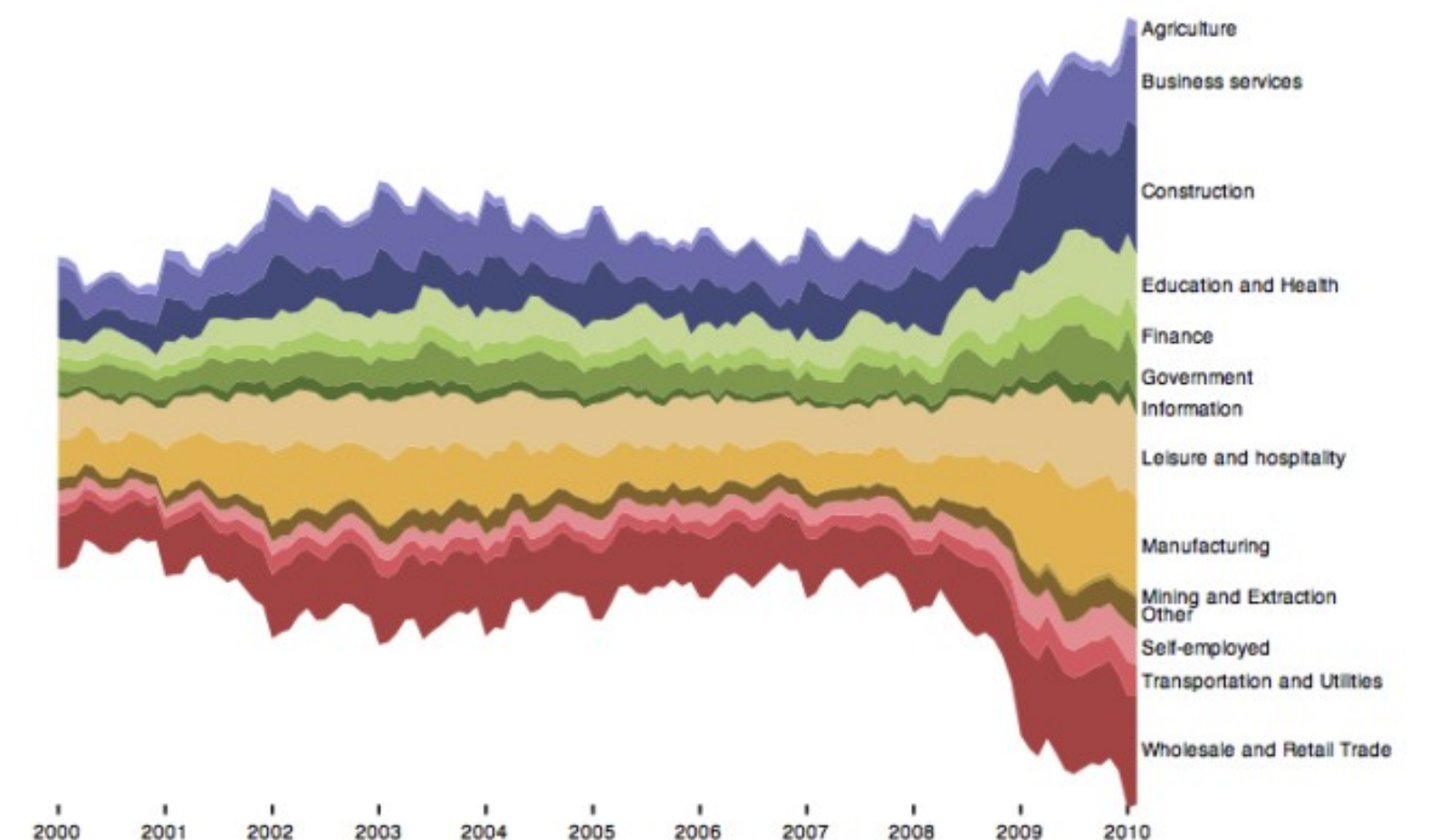
Some are common, some are “exotic”, thus it is a tour through “the zoo”

Index Charts



With some forms of time-series data, raw values are less important than relative changes. Consider investors, who are more interested in a stock's growth rate than its specific price. Multiple stocks may have dramatically different baseline prices, but be meaningfully compared when normalized. An *index chart* is an interactive line chart that shows percentage changes for a collection of time-series based on a selected index point. For example, the image above shows the percentage change of selected stock prices if purchased in January 2005: one can see the rocky rise enjoyed by those who invested in Amazon, Apple, or Google at that time.

Stacked Graphs



Jacques Bertin

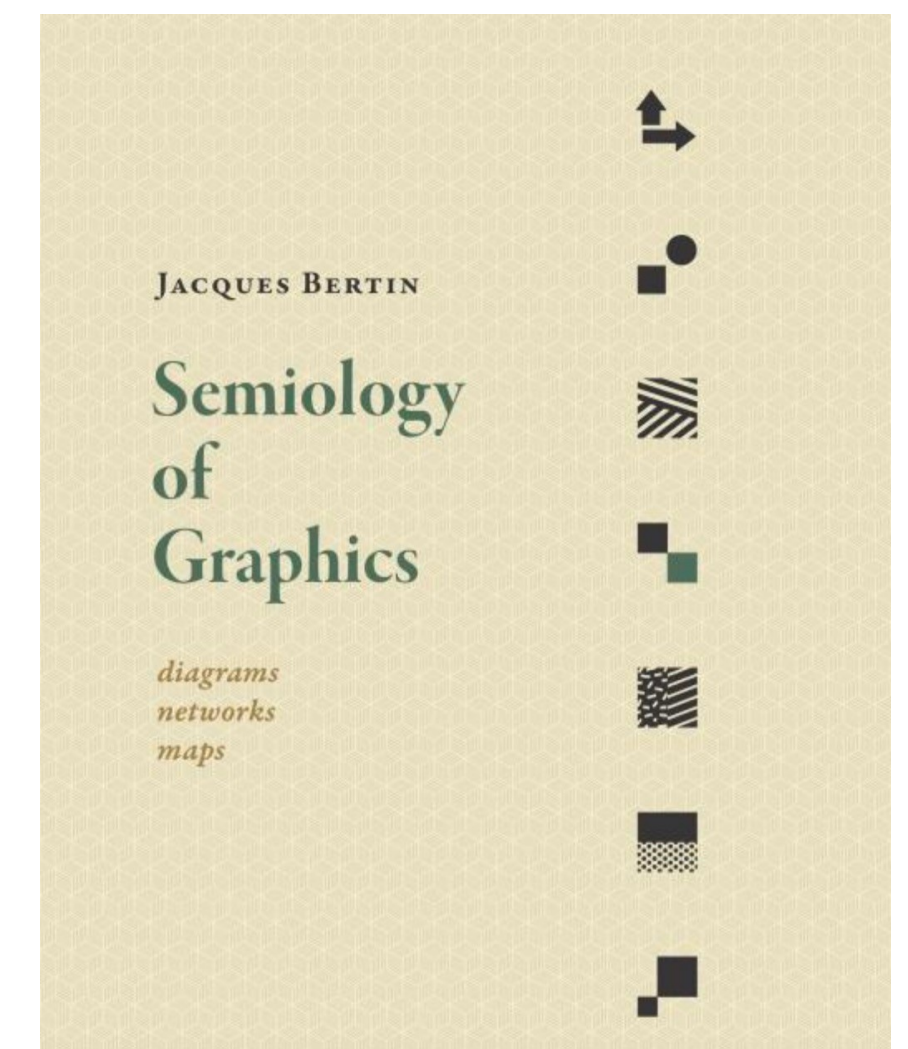
Visual Language is a Sign System

Images perceived as a set of signs

Sender encodes information in signs

Receiver decodes information from signs

Sémiologie Graphique, 1967



The Big Picture



Physical Data Types

int, float, string

Conceptual Data Types

temperature, location

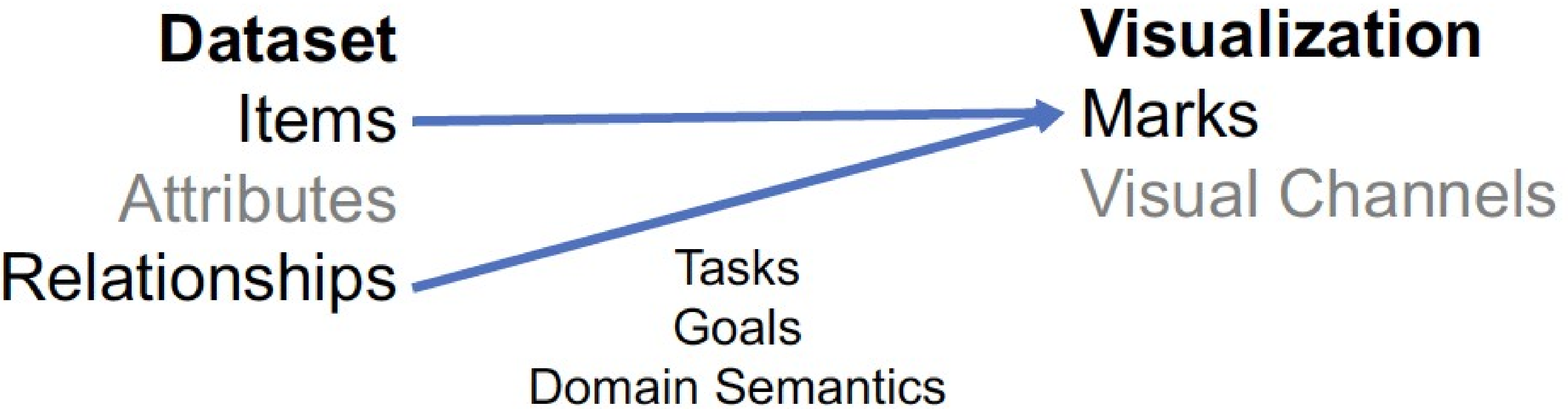
Visual Channels

x, y, color, opacity

Graphical Marks

rect, line, point, area

Goal of Visual Encoding



Mark

Aka visual object, graphical object

Basic graphical element in an image

Geometric primitive

- 0D: a point
- 1D: a line/path
- 2D: an area
- 3D: a volume (not frequently used)

Marks as Items

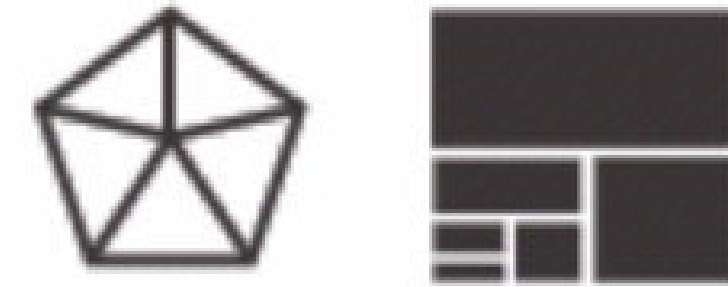
➔ Points



➔ Lines

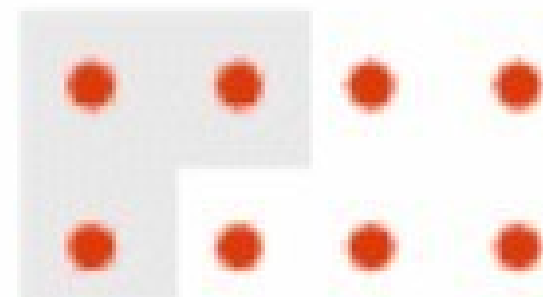


➔ Areas

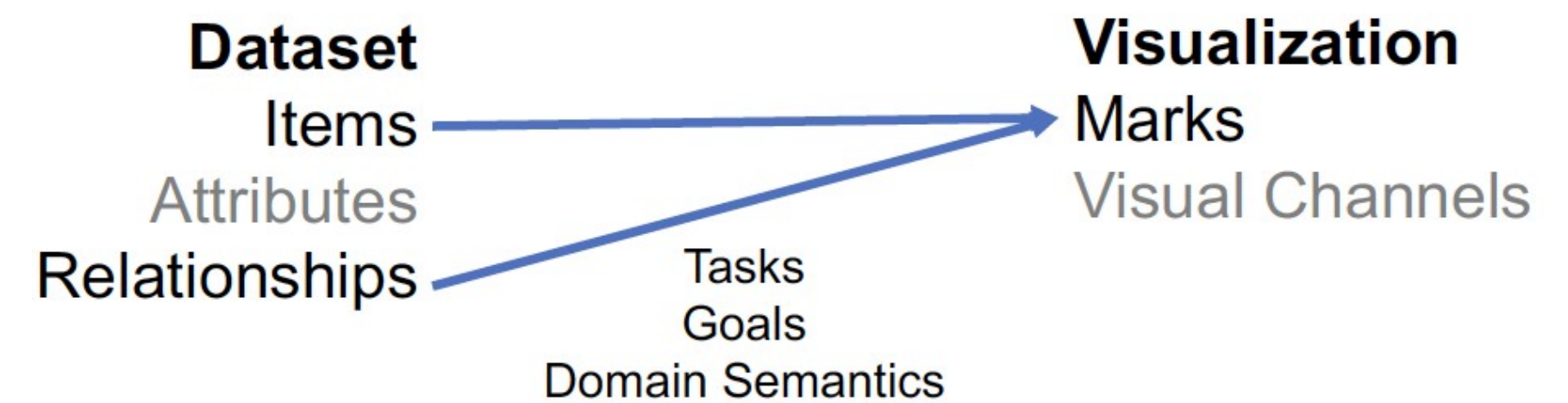
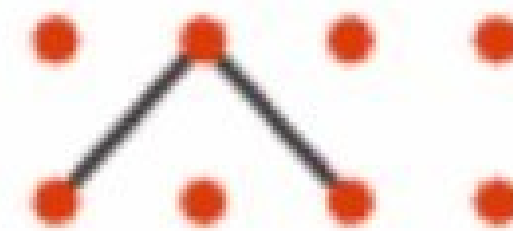


Marks as Relationships

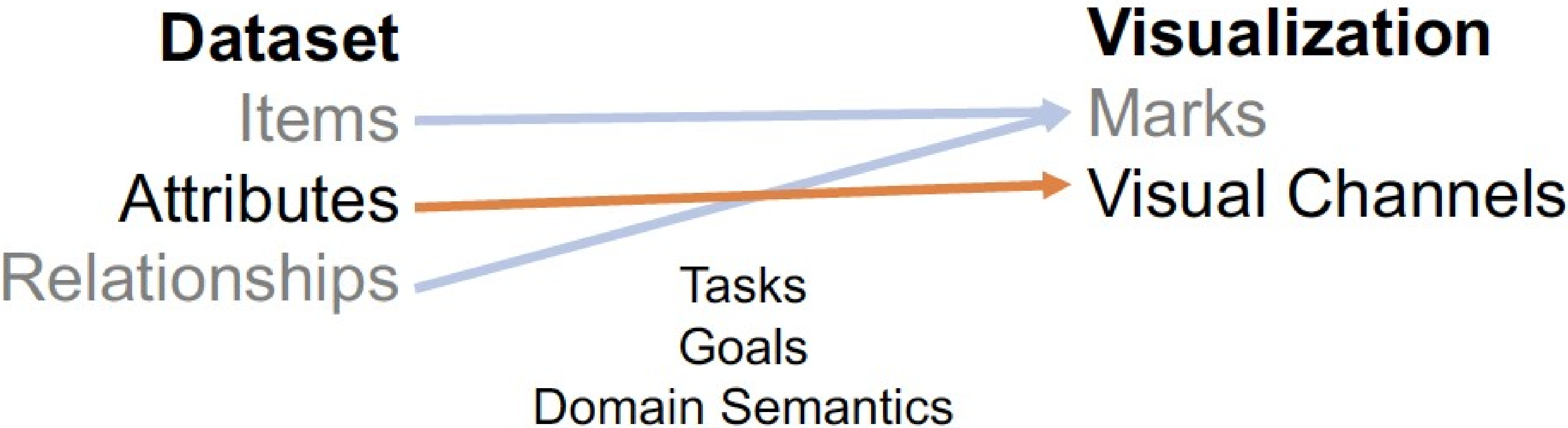
➔ Containment



➔ Connection



Goal of Visual Encoding



Visual Channel

Aka visual variable, retinal variable, visual attribute

A way to control the appearance of marks

➔ Position

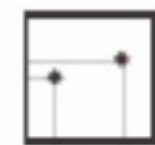
➔ Horizontal



➔ Vertical



➔ Both



➔ Color (Hue, Saturation, Value)



➔ Shape



➔ Tilt

/Angle/Orientation



➔ Size

➔ Length



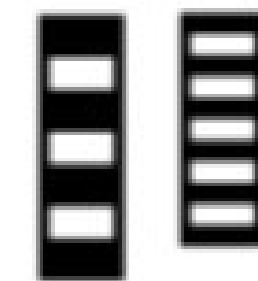
➔ Area



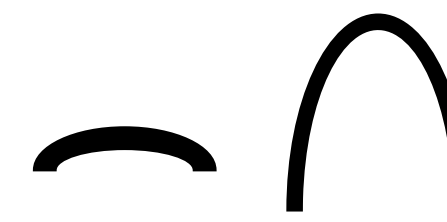
➔ Volume



Texture



Curvature



Motion Pattern

Color: Hue, Saturation, and Value

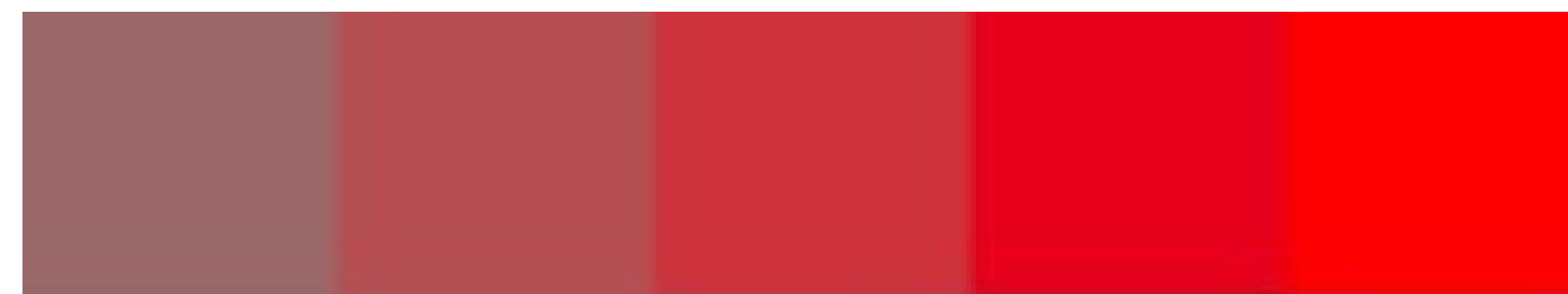
- Value: the brightness of a color



- Hue: what we typically mean by "color"



- Saturation: the purity of a color (dominance of hue in the color)



Types of Channels

Magnitude Channels

How much?

Position

Length

Saturation ...

Ordinal & Quantitative Data

Identity Channels

What? Where?

Shape

Color (hue)

Spatial region ...

Nominal Data

Modernization of Visual Channels

➔ **Magnitude Channels:** **O** or **Q** attributes



➔ **Identity Channels:** **N** attributes



Tamara Munzner, *Visualization Analysis and Design* (2014).

A visual encoding defines

- A mapping between data items and marks
- A mapping between data attributes and visual channels
- Scale: type, domain, range, aggregator

Chart Types: A Deconstruction Approach

- Number of data variables
- Data variable types
- Visual encodings

Univariate Data

value
9
13
8
96
88
56
...

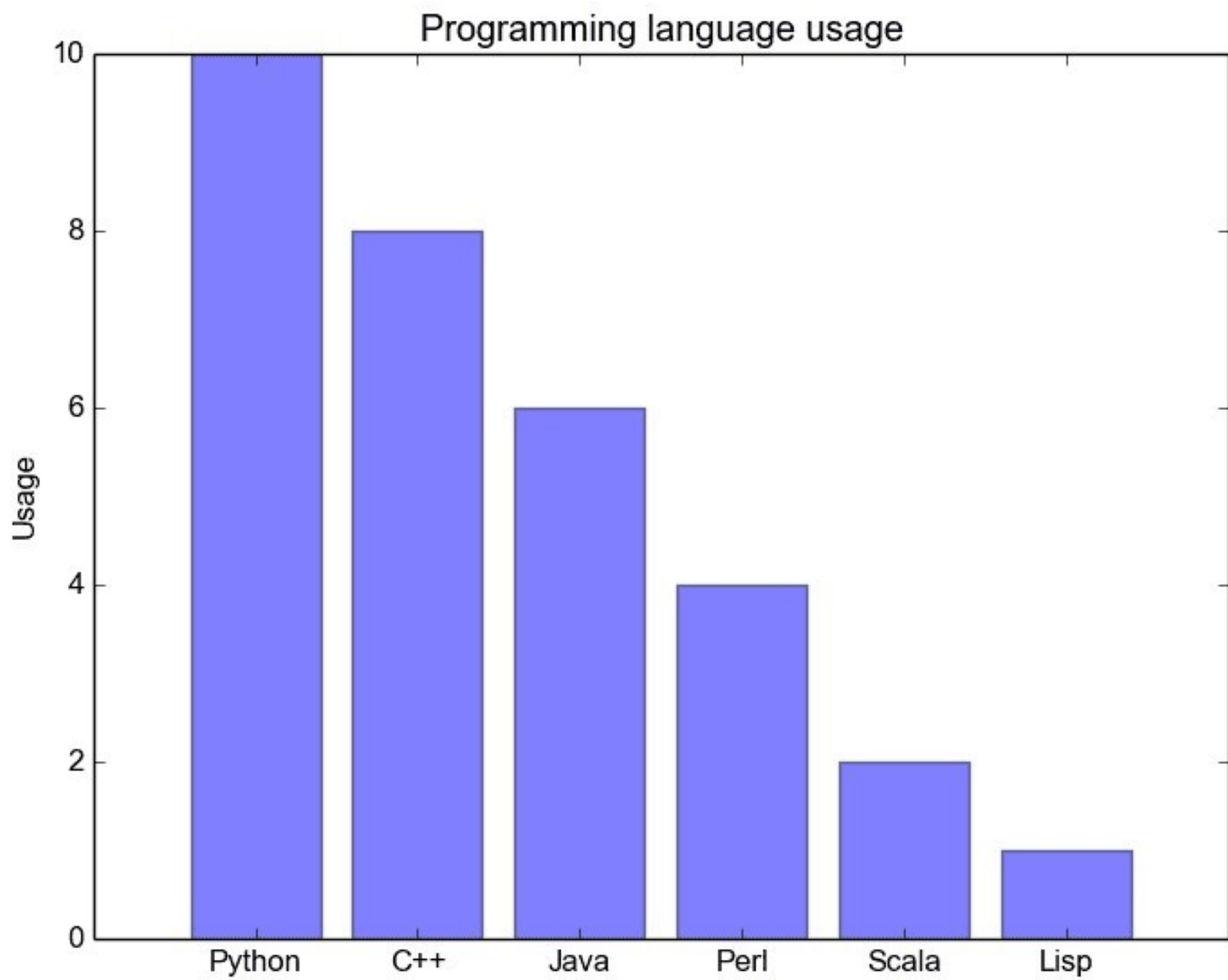


What mark[s] and encodings are used in this chart?

Point Mark: a value item
X position: Q

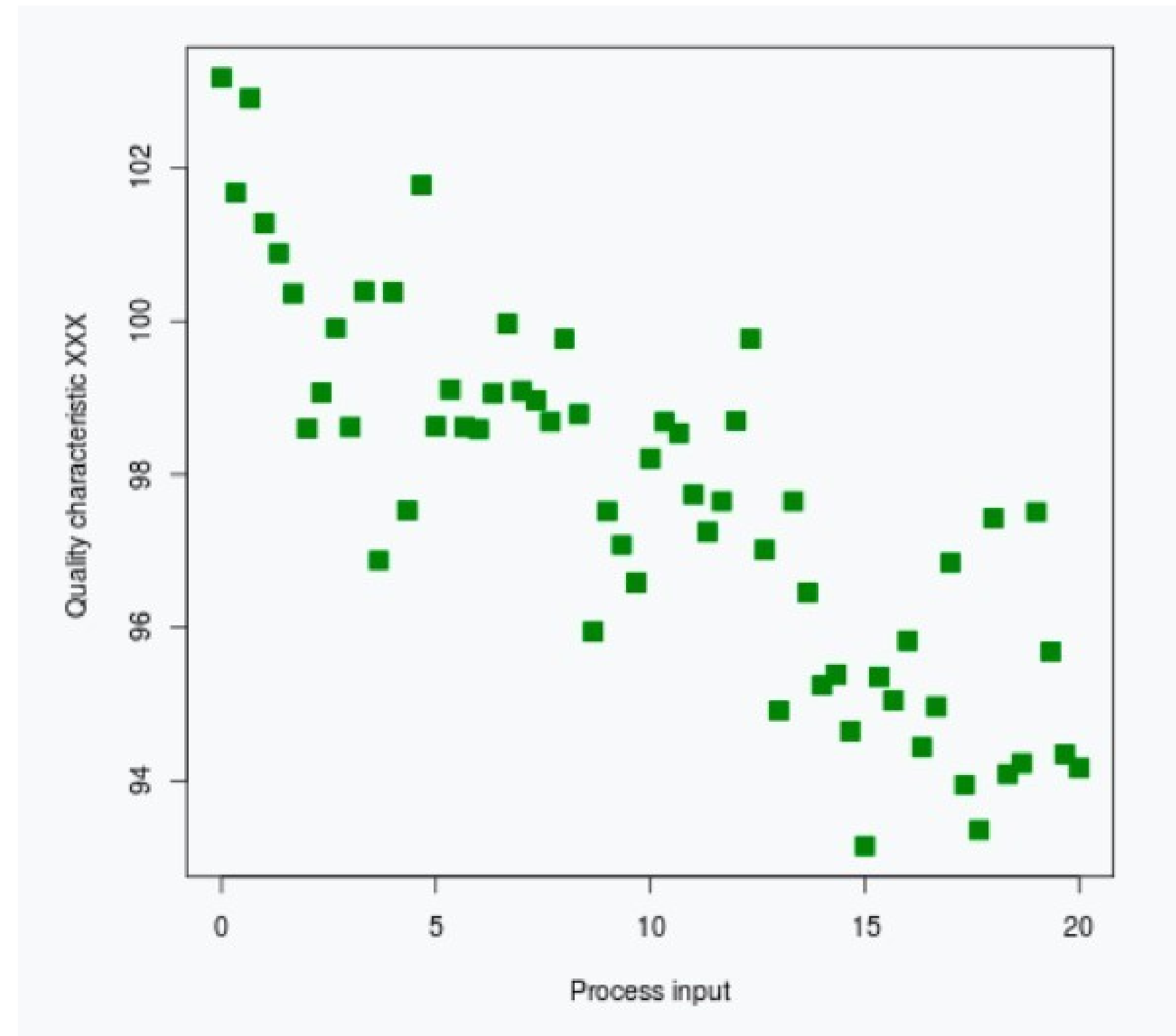
Bivariate Data: Bar Chart

name	value
n1	9
n2	13
n3	8
n4	96
n5	88
n6	56
...	...



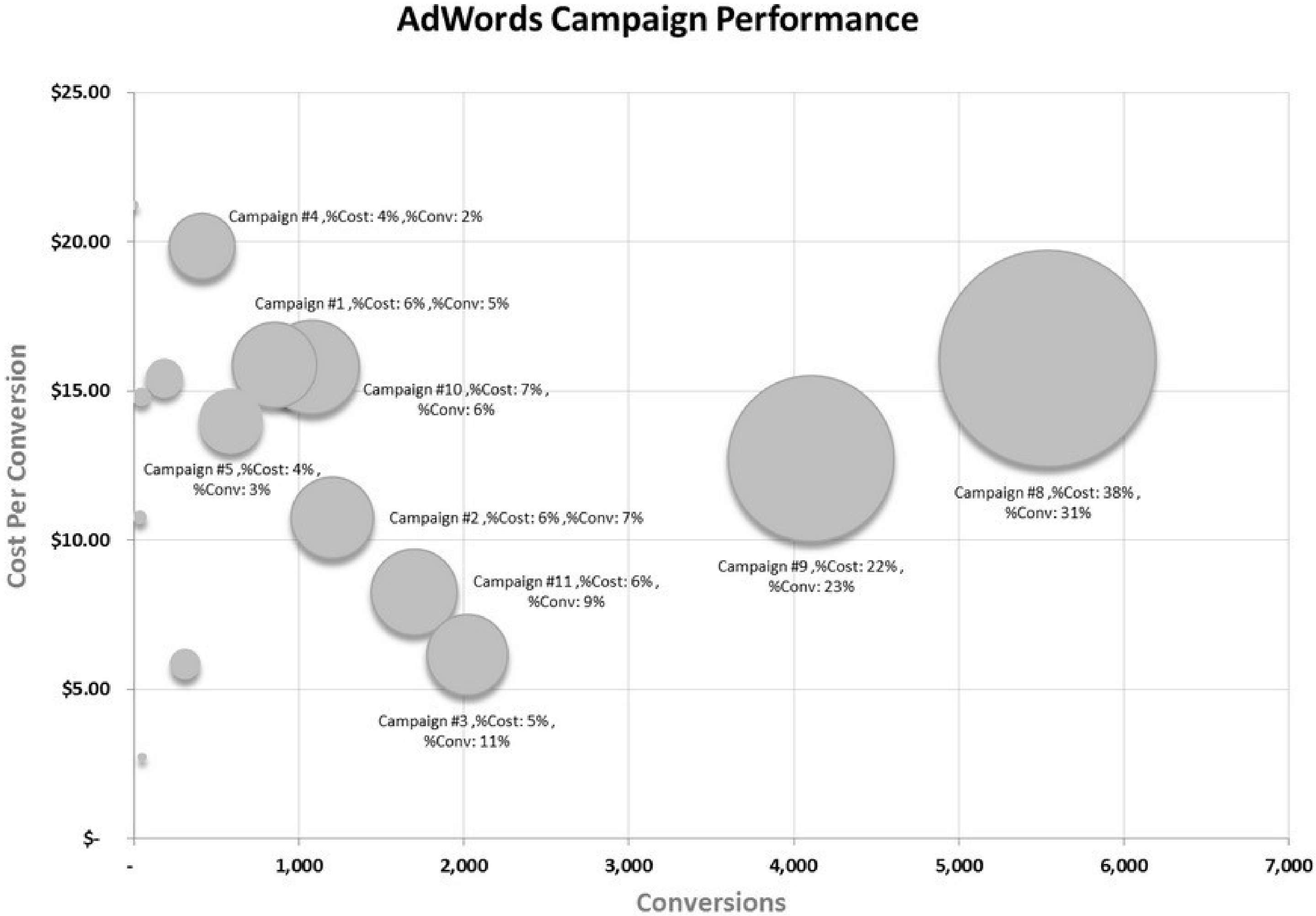
Bar Mark: an item
X position: N
Y position: Q

Bivariate Data: Scatter Plot

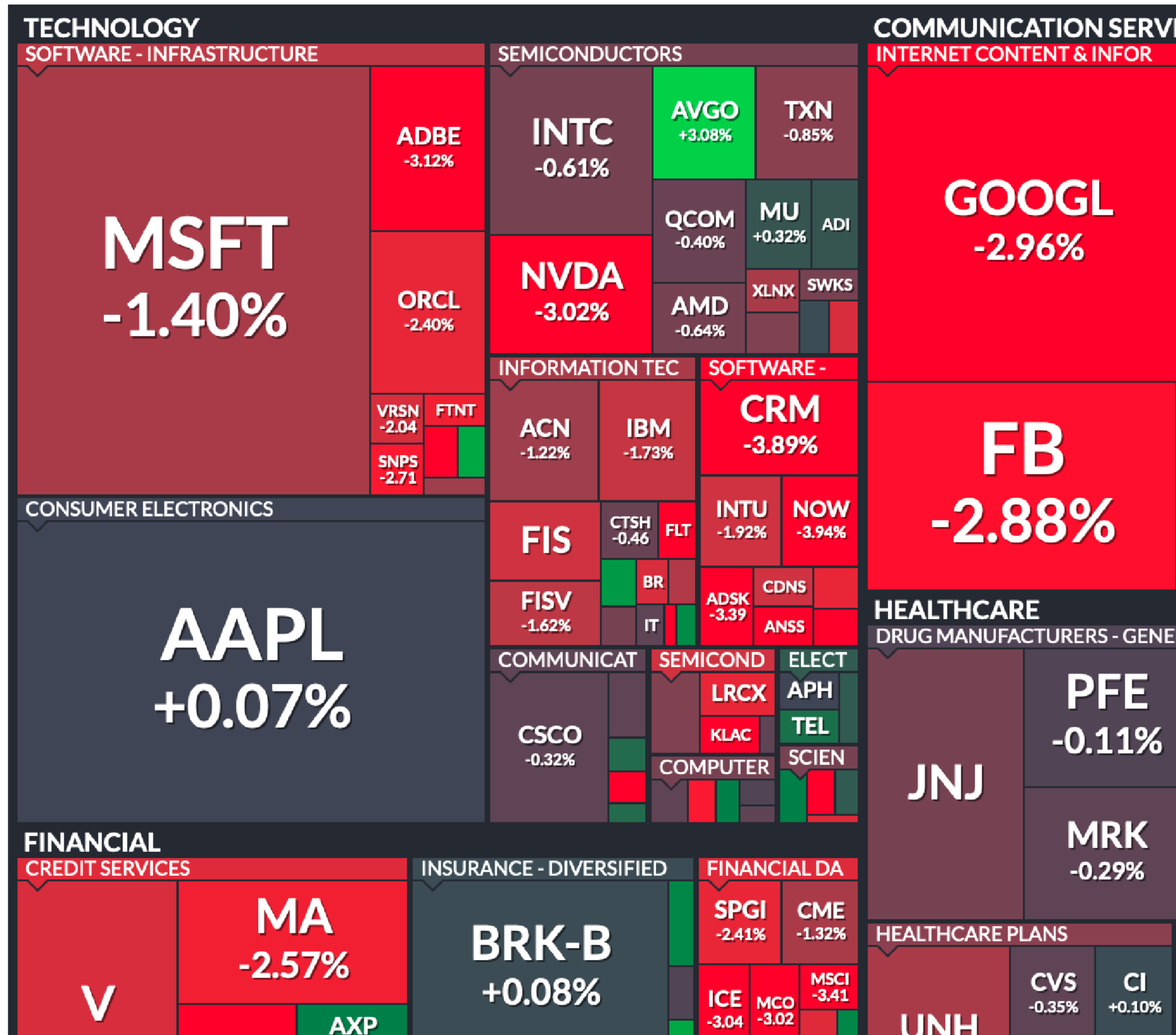


Point Mark: an item
X position: Q
Y position: Q

Trivariate Data ?



Mark: an item
X position: Q
Y position: Q
Size: Q



Rectangle Mark: item & Containment

Area: Market cap (Q)

Rectangle Position: Market sector (N), market cap (Q)

Color Hue: loss vs. gain (N, O)

Color value: magnitude of loss or gain (Q)

Deconstructing charts

Walk around the room and look at the charts.

Write out **one mapping**:

- A mapping between data items and marks
- A mapping between data attributes and visual channels

If all mappings are already written out by other students, write your answer to the following about a particular mapping: is it a good one?

A visual encoding defines

- A mapping between data items and marks
- A mapping between data attributes and visual channels
- Scale: type, domain, range, aggregator

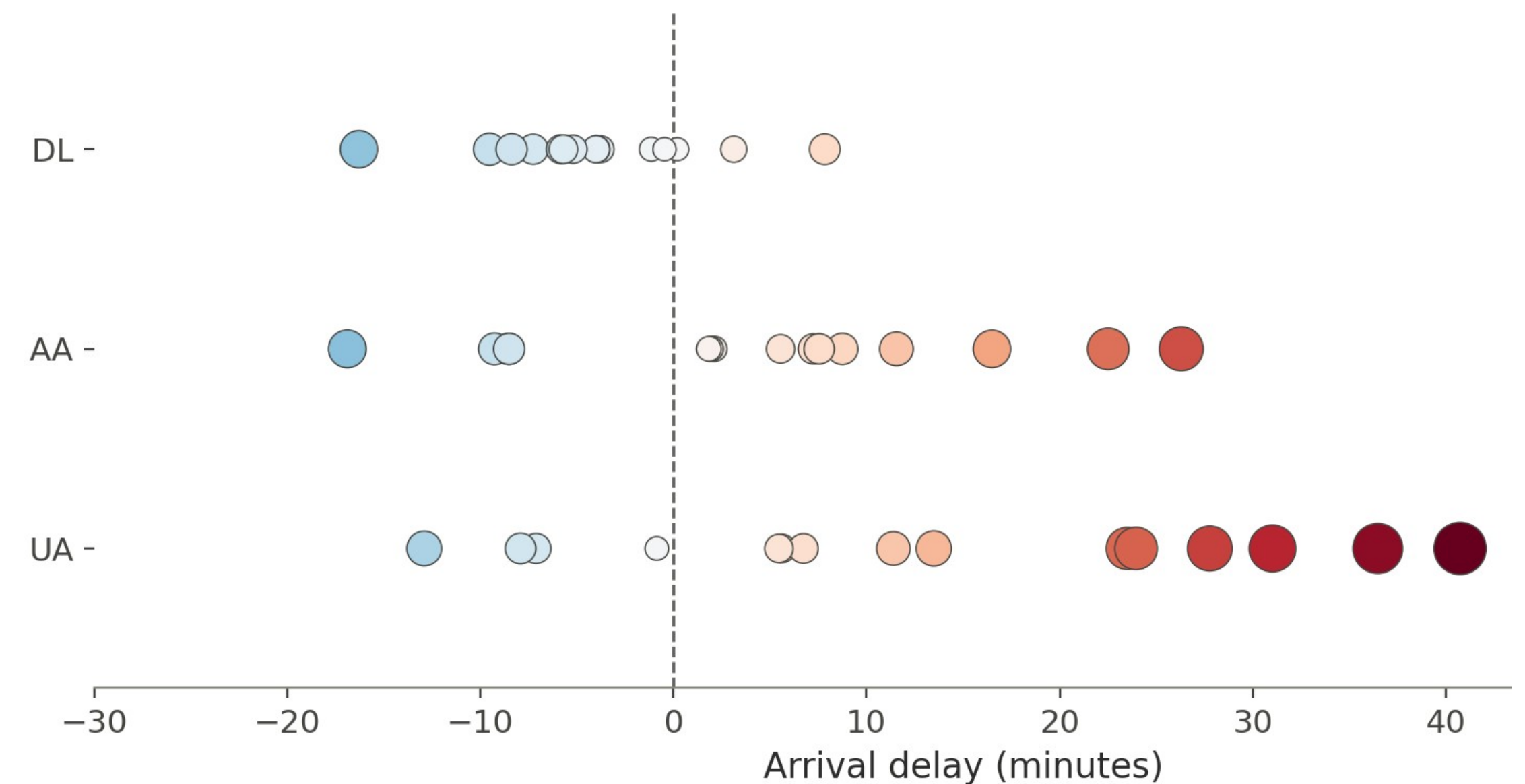
What is a scale?

A function specifying **how** abstract data values are mapped to visual properties

Dimensions of a scale

- **Domain:** the data values to be mapped

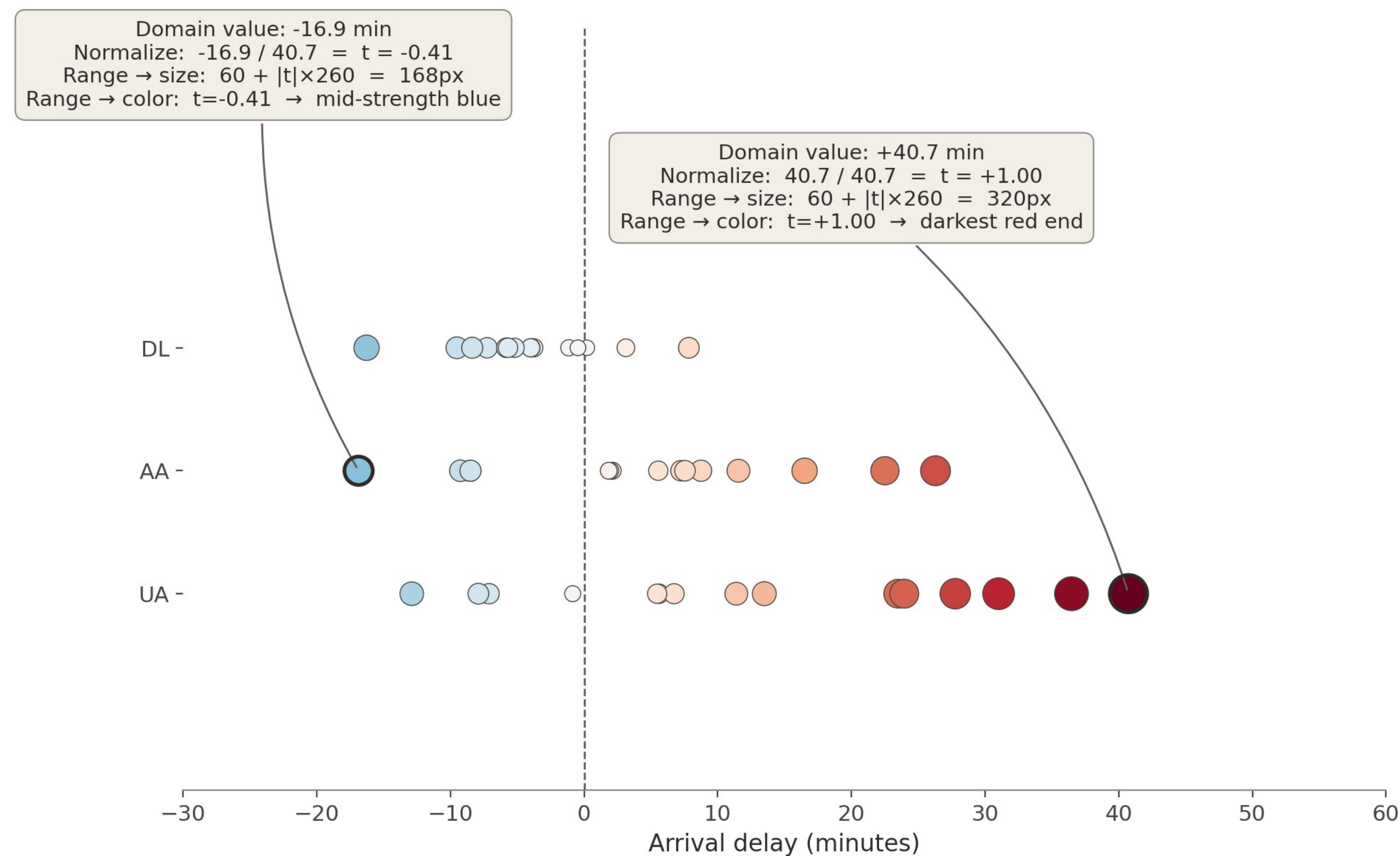
Airlines = [DL, AA, UA]
Arrival delay = [-30,40]



Dimensions of a scale

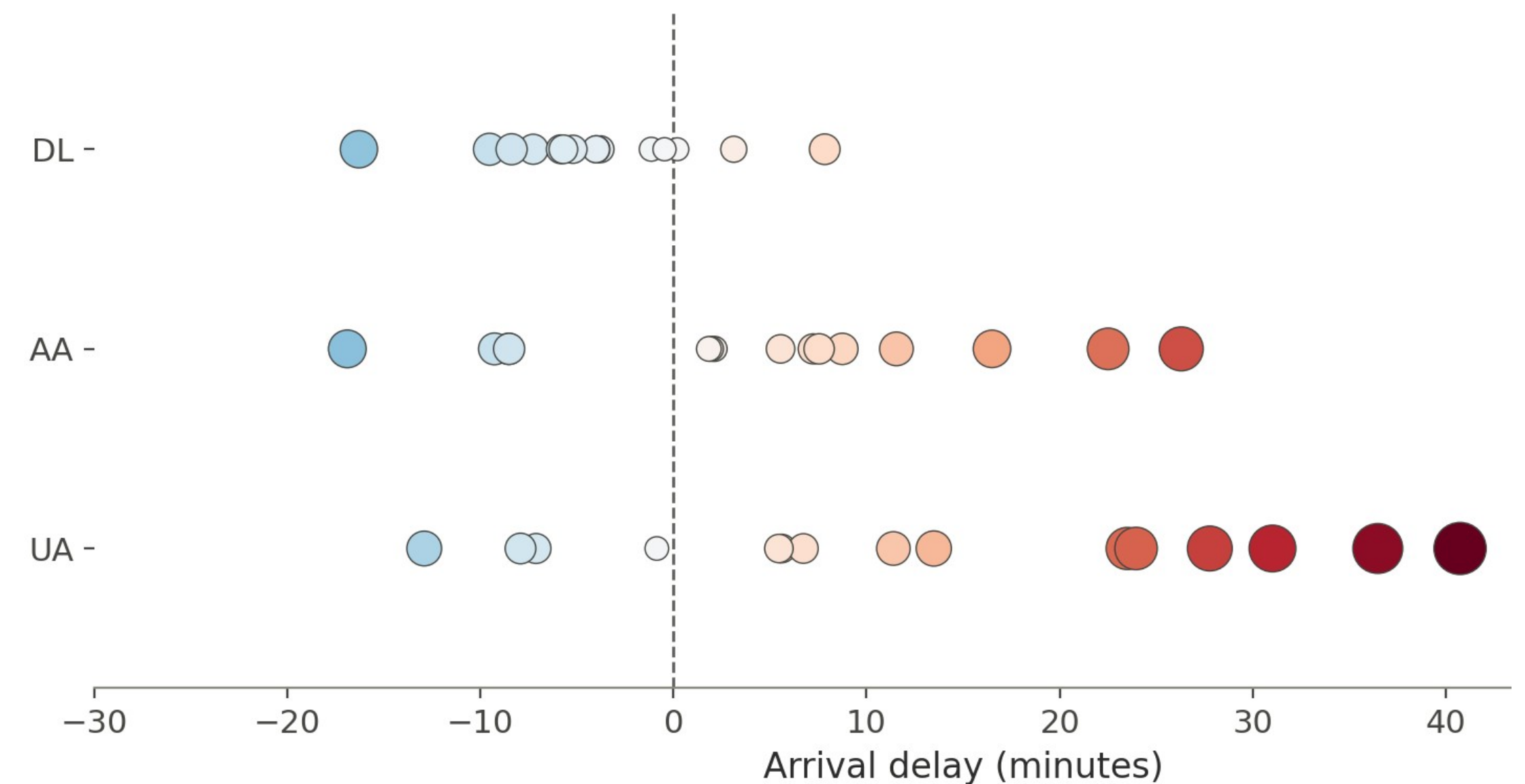
- **Domain:** the data values to be mapped
- **Range:** the visual channel values derived from domain

Airlines = [DL, AA, UA]
Arrival delay = [-30,40]



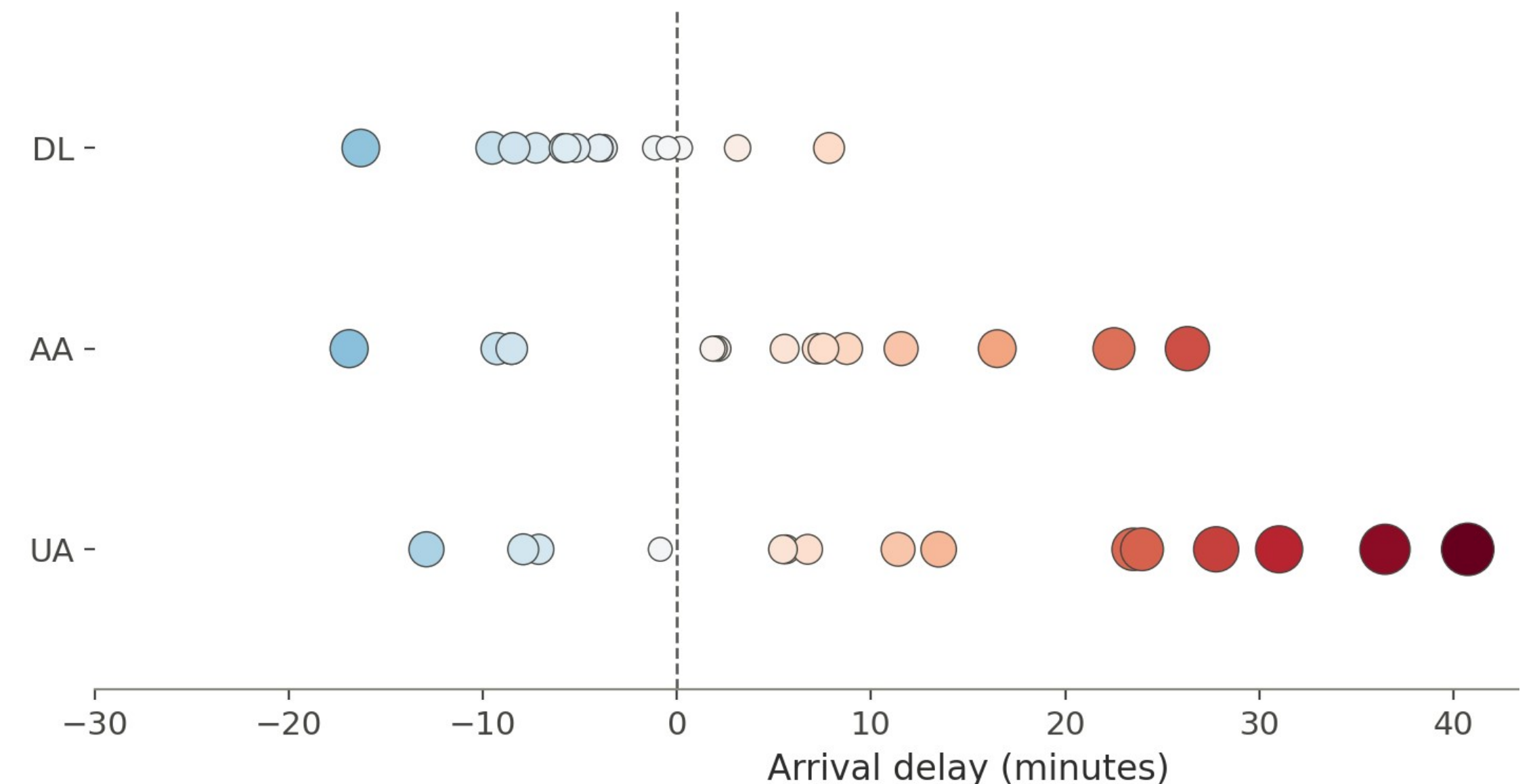
Dimensions of a scale

- **Domain:** the data values to be mapped
- **Range:** the visual channel values derived from domain
- **Type:** sequential, diverging, linear, log, power, square root, ...



Dimensions of a scale

- **Domain:** the data values to be mapped
- **Range:** the visual channel values derived from domain
- **Type:** sequential, diverging, linear, log, power, square root, ...
- **Aggregator:** how values are aggregated if a mark represents multiple items



Guidelines: Color Scales for Quantitative Data

Sequential scale: ordered values

Diverging scale: Ordered values increasing or decreasing from a mid point

Sequential color scale

You can still use hues in addition to value/lightness

`# d3.interpolateViridis(t) <>`



Given a number t in the range $[0,1]$, returns the corresponding color from the “viridis” perceptually-uniform color scheme designed by [van der Walt, Smith and Firing](#) for matplotlib, represented as an RGB string.

`# d3.interpolateInferno(t) <>`



Given a number t in the range $[0,1]$, returns the corresponding color from the “inferno” perceptually-uniform color scheme designed by [van der Walt and Smith](#) for matplotlib, represented as an RGB string.

`# d3.interpolateMagma(t) <>`



Given a number t in the range $[0,1]$, returns the corresponding color from the “magma” perceptually-uniform color scheme designed by [van der Walt and Smith](#) for matplotlib, represented as an RGB string.

`# d3.interpolatePlasma(t) <>`



Given a number t in the range $[0,1]$, returns the corresponding color from the “plasma” perceptually-uniform color scheme designed by [van der Walt and Smith](#) for matplotlib, represented as an RGB string.

`# d3.interpolateCividis(t) <>`



Diverging color scale

```
# d3.interpolateBrBG(t) <>  
# d3.schemeBrBG[k]
```



Given a number t in the range $[0,1]$, returns the corresponding color from the “BrBG” diverging color scheme represented as an RGB string.

```
# d3.interpolatePRGn(t) <>  
# d3.schemePRGn[k]
```



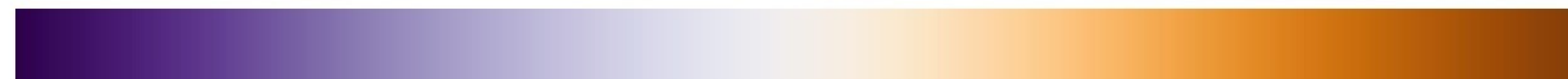
Given a number t in the range $[0,1]$, returns the corresponding color from the “PRGn” diverging color scheme represented as an RGB string.

```
# d3.interpolatePiYG(t) <>  
# d3.schemePiYG[k]
```



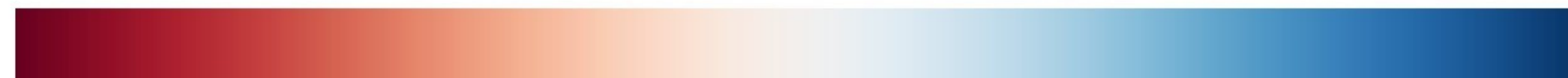
Given a number t in the range $[0,1]$, returns the corresponding color from the “PiYG” diverging color scheme represented as an RGB string.

```
# d3.interpolatePuOr(t) <>  
# d3.schemePuOr[k]
```



Given a number t in the range $[0,1]$, returns the corresponding color from the “PuOr” diverging color scheme represented as an RGB string.

```
# d3.interpolateRdBu(t) <>  
# d3.schemeRdBu[k]
```

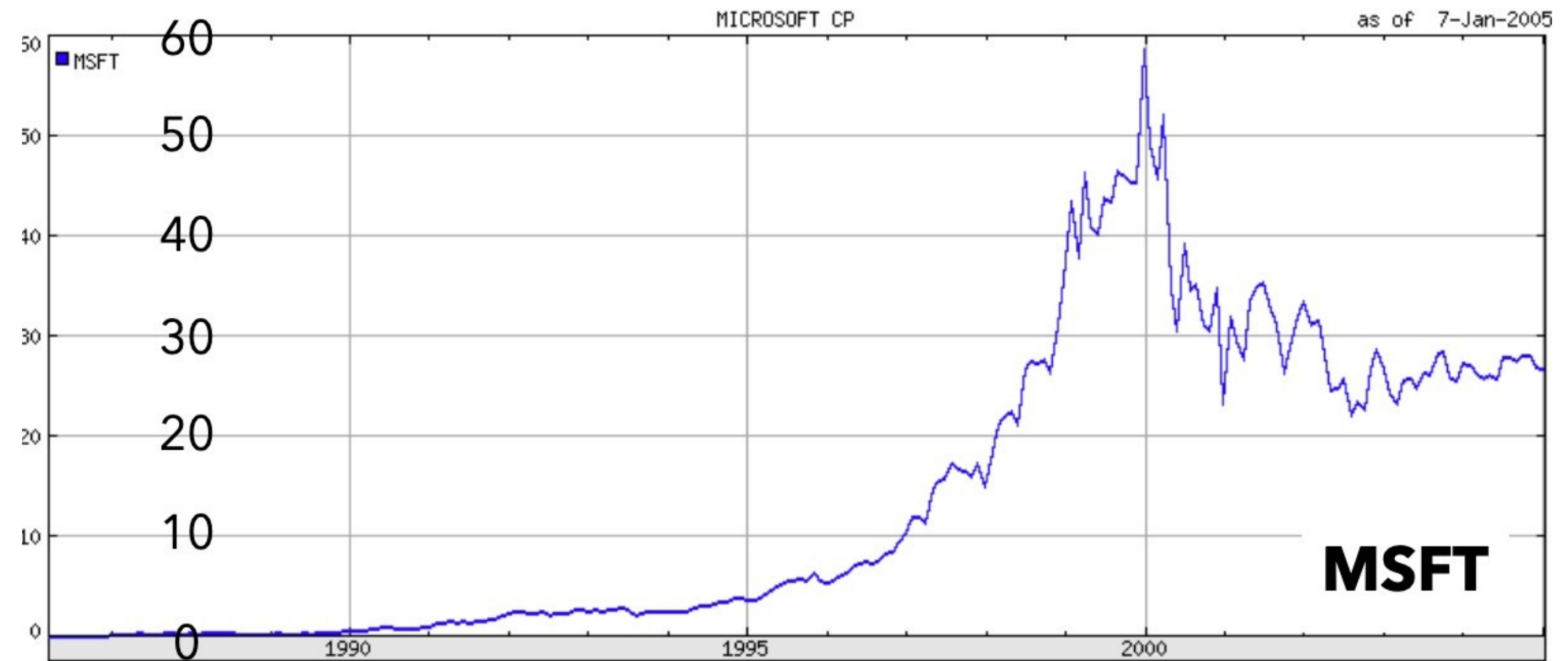


Guidelines: Scales for Position/Length Encodings

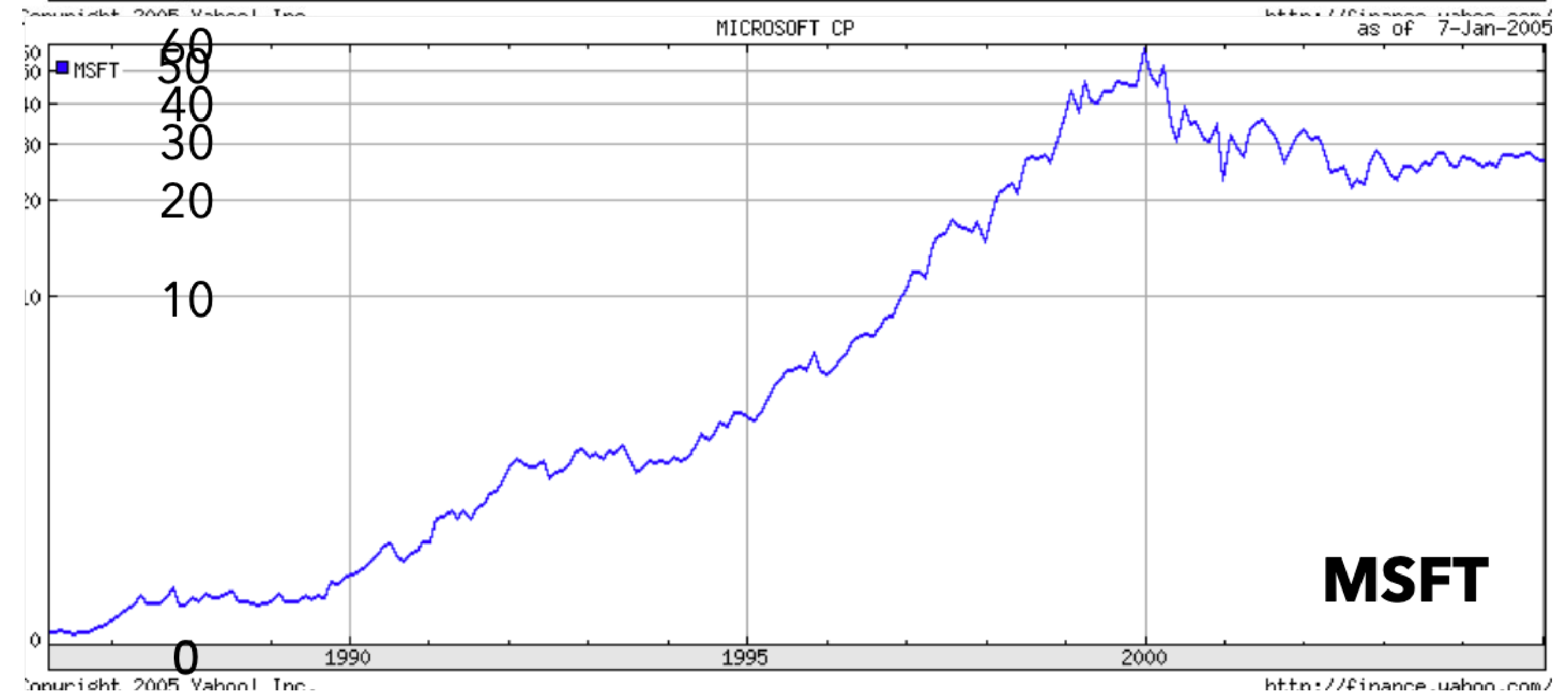
Design Considerations

- Linear vs. Log Scale
- Starting from Zero
- Outliers and Scale Break
- Axis Tick Mark

Linear Scale
Absolute Change



Log Scale
Small fluctuations
Percent change



When to apply a log scale?

- Address data skew (e.g., long tails, outliers)
Enables comparison within and across multiple orders of magnitude.
- Focus on multiplicative factors (not additive) Recall that the logarithm transforms $\times$ to $+$! Percentage change, not absolute value.
- Constraints
positive, non-zero values
audience familiarity

Guidelines: Scales for Position/Length Encodings

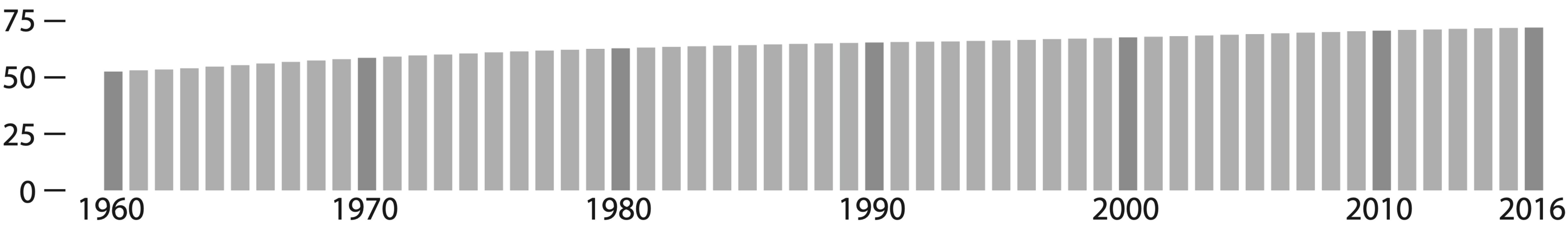
Design Considerations

- Linear vs. Log Scale
- Starting from Zero
- Outliers and Scale Break
- Axis Tick Mark

All Charts Should Start at Zero

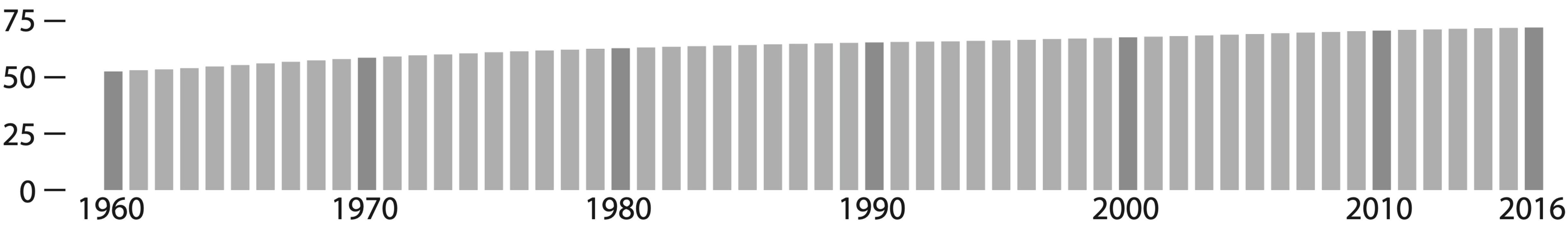
Myth or Truth?

Average world life expectancy at birth (years)



(Source: World Bank Data)

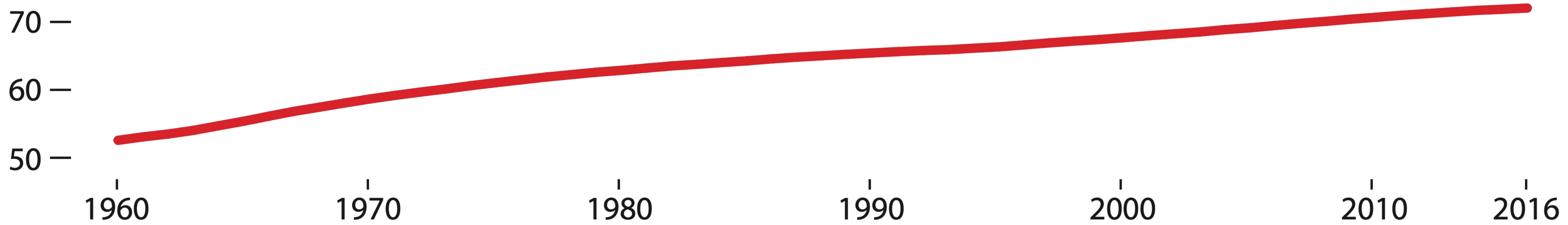
Average world life expectancy at birth (years)

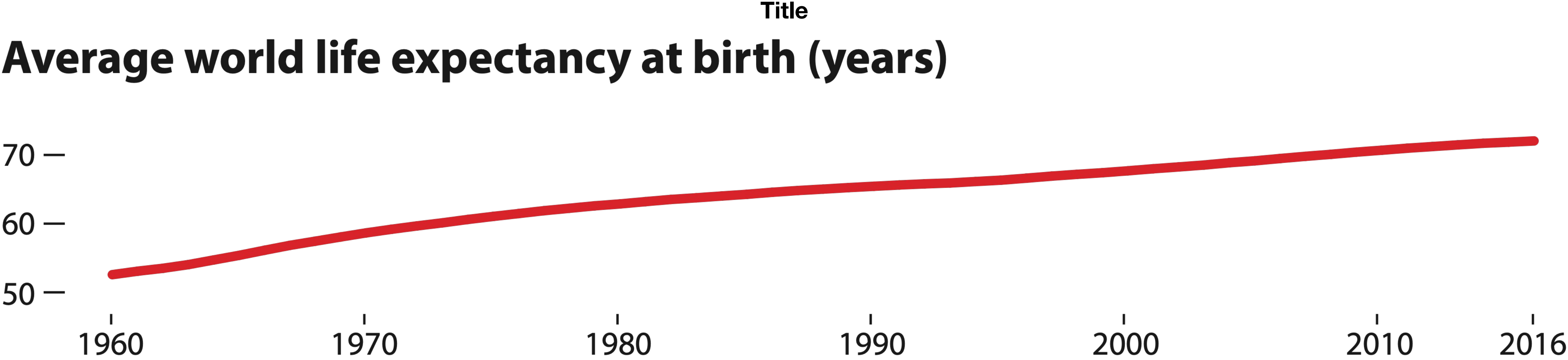


(Source: World Bank Data)

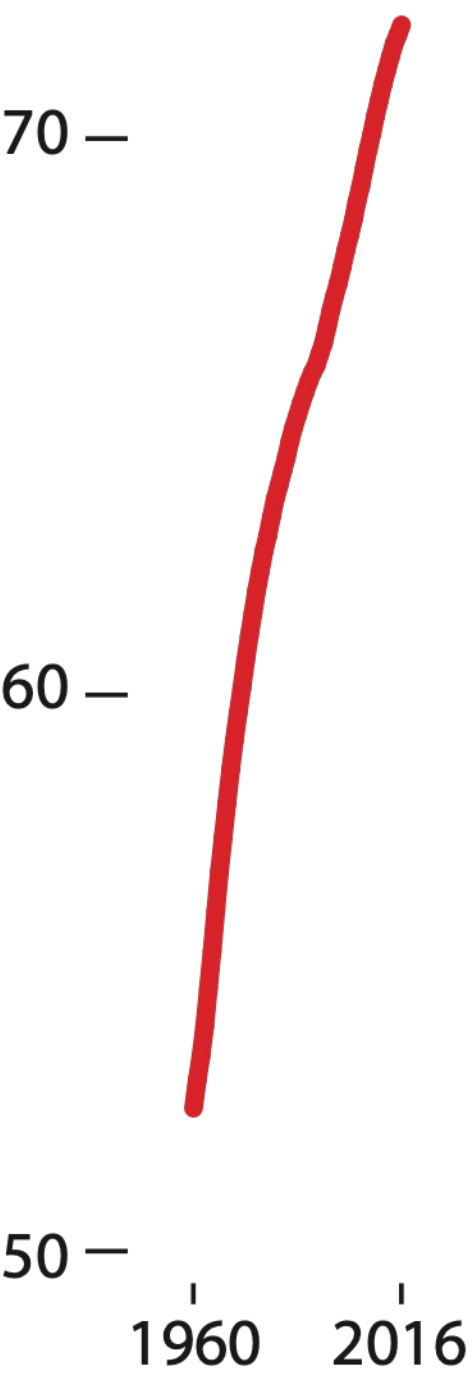
Title

Average world life expectancy at birth (years)

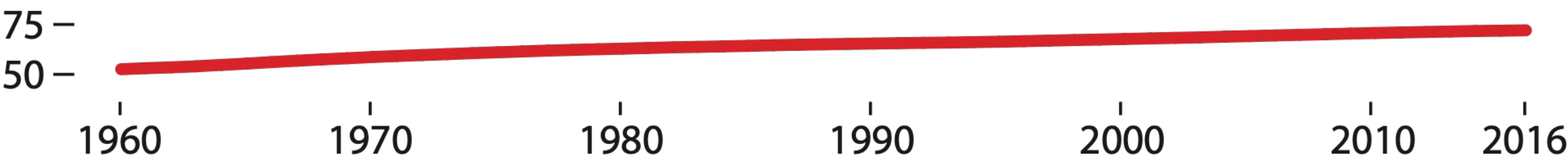




**Too narrow
and tall**

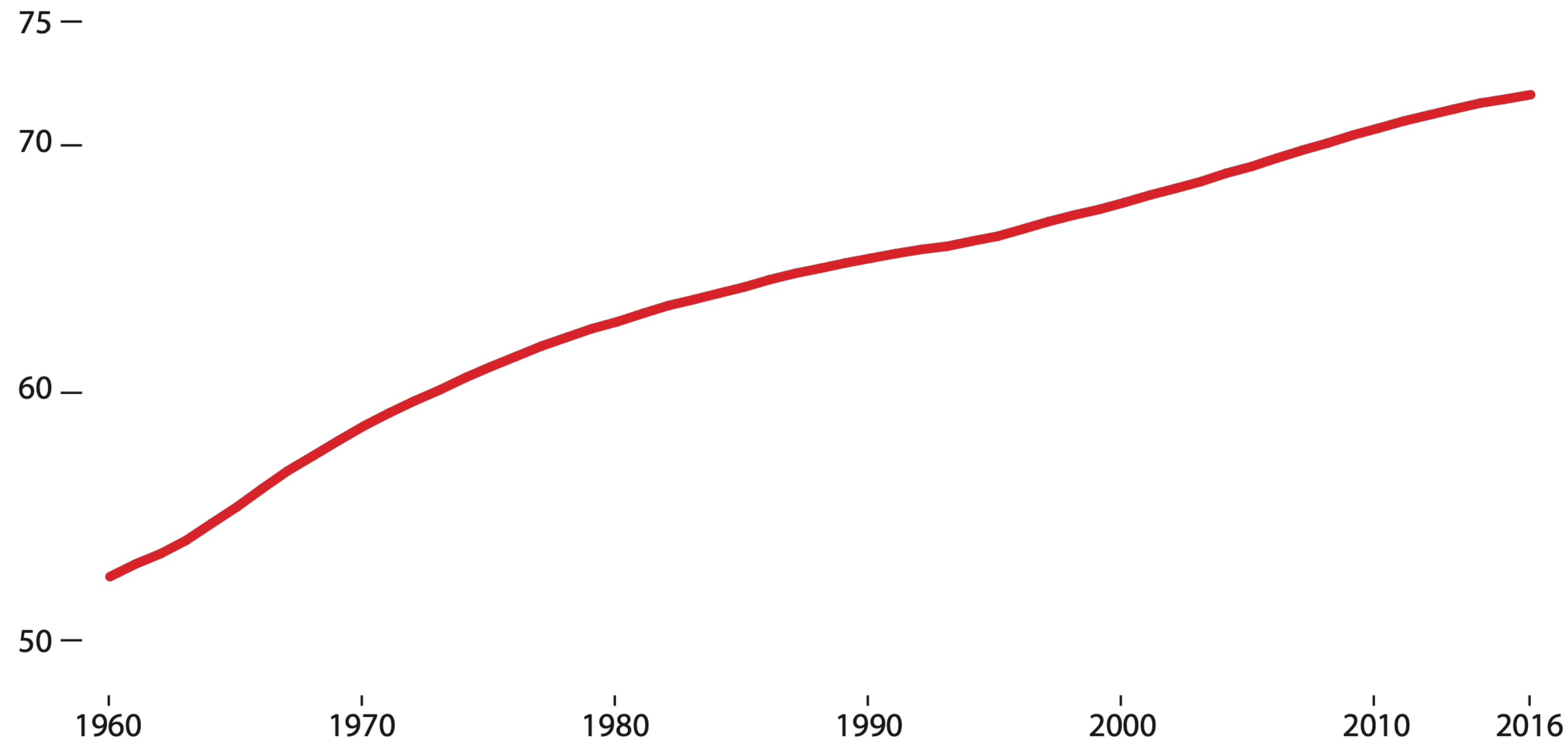


Too wide and short



Aspect Ratio

Average world life expectancy at birth (years)



Approximate the proportion of the chart to match the depicted trend

35% increase $\approx 1/3_{\text{rd}}$
 $\approx 3:1$ aspect ratio

Guideline if relevant data are far from zero

Center the y-axis on the data's mean, then extend the y-axis 0.75 SD above and below the mean (Witt 2019)

Guidelines: Scales for Position/Length Encodings

Design Considerations

- Linear vs. Log Scale
- Starting from Zero
- Outliers and Scale Break
- Axis Tick Mark

userratio

timetaken

150

100

50

0

10

15

25

30

35

38

45

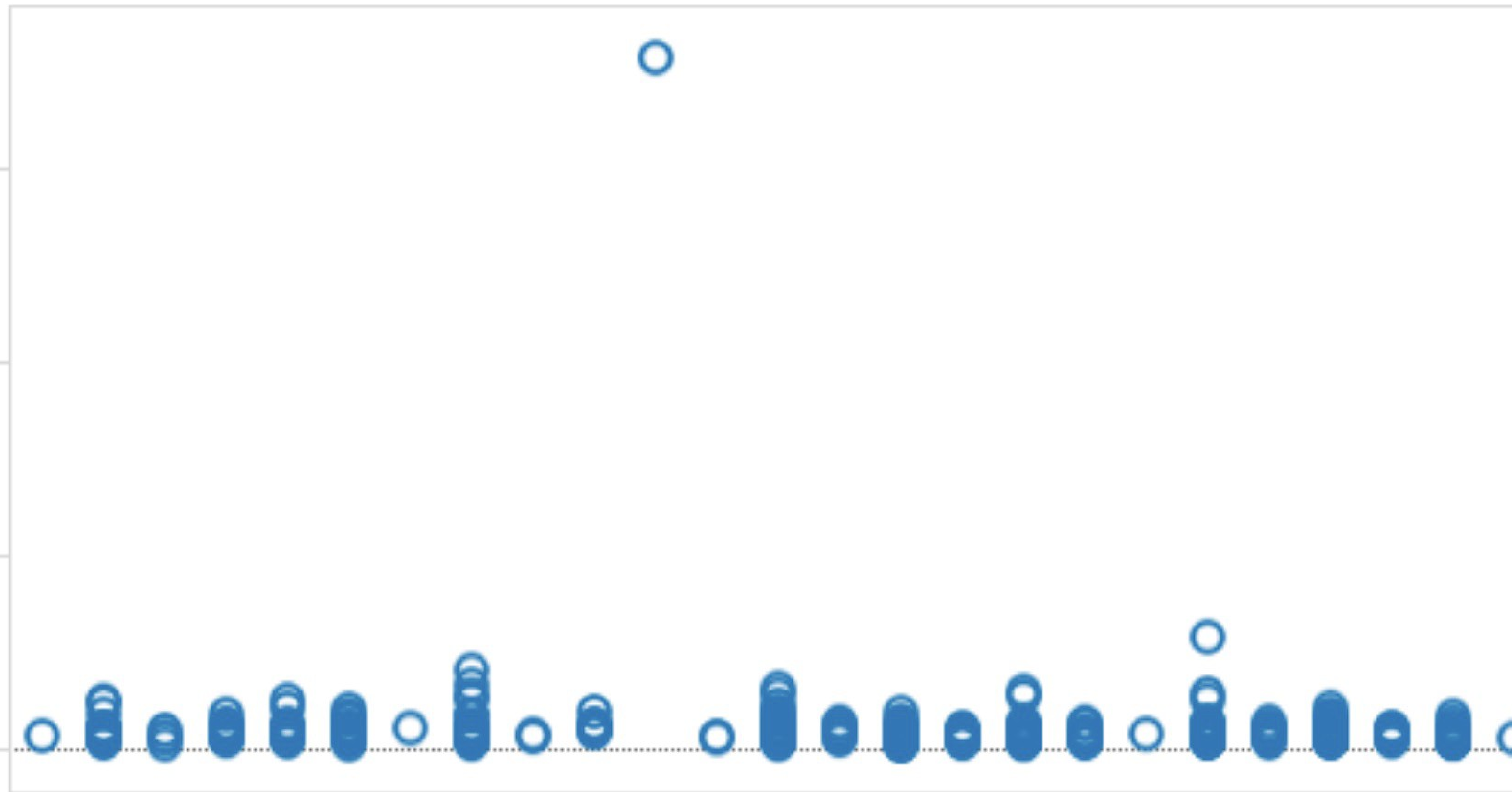
55

65

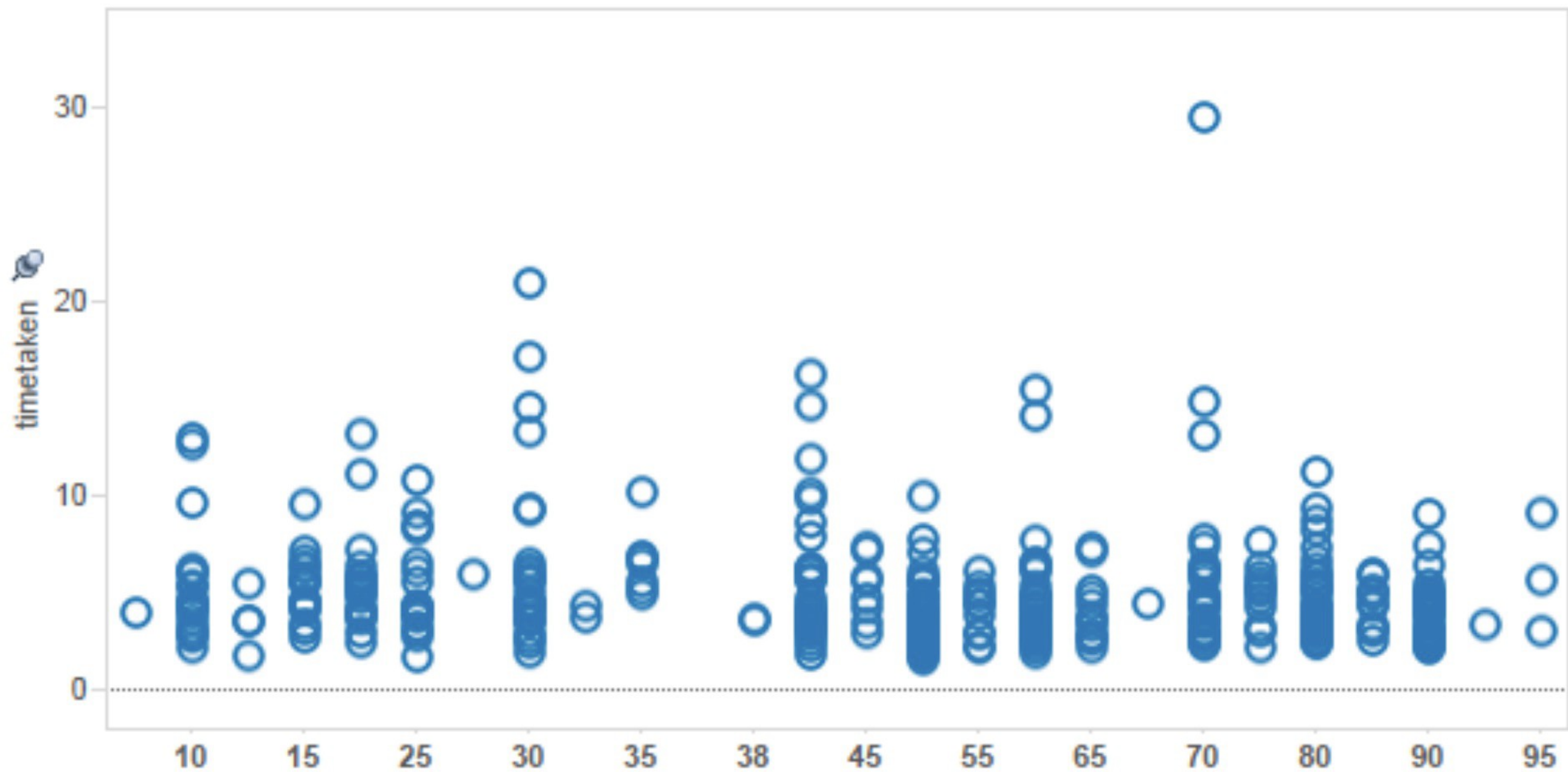
70

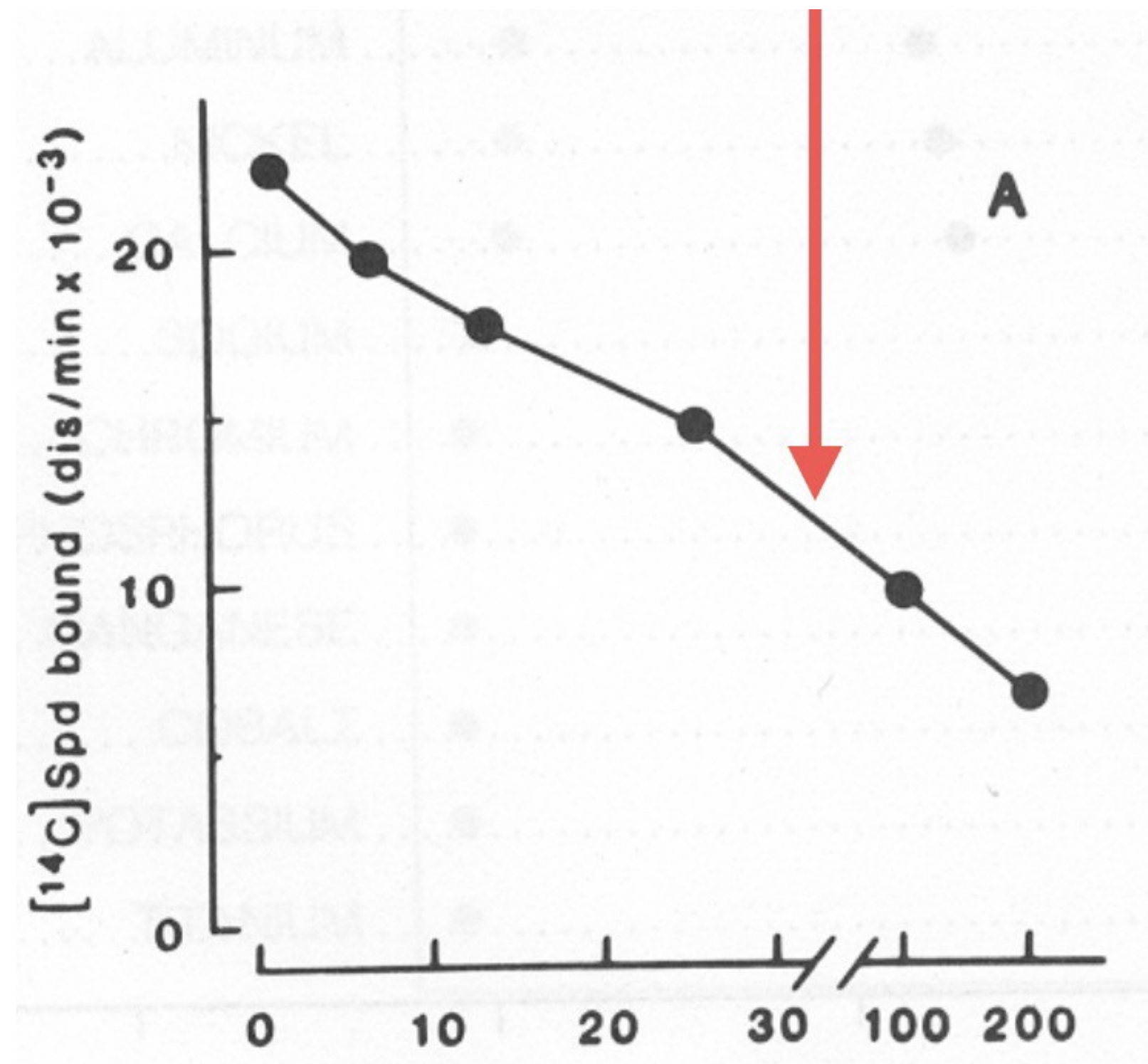
80

90

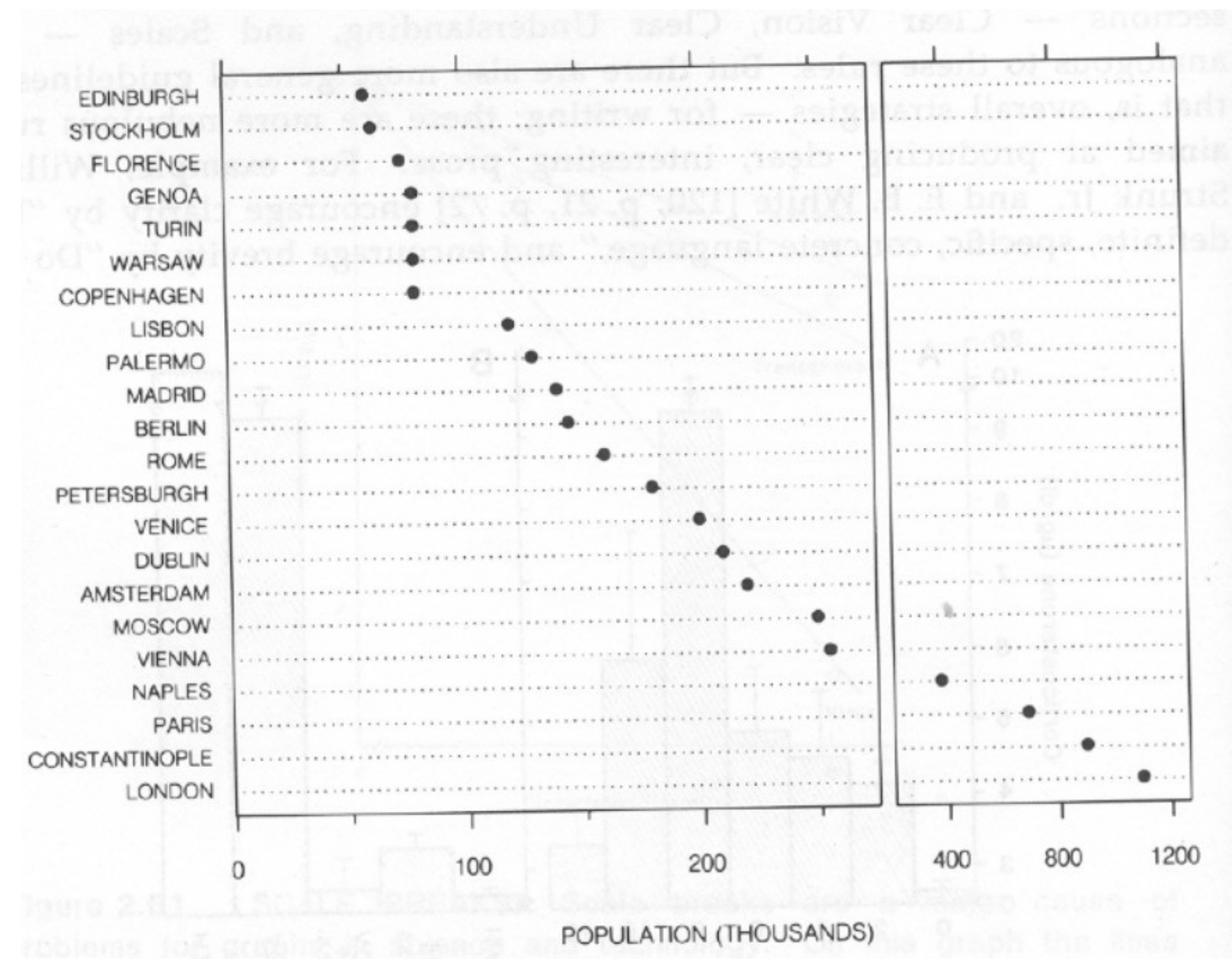


userratio

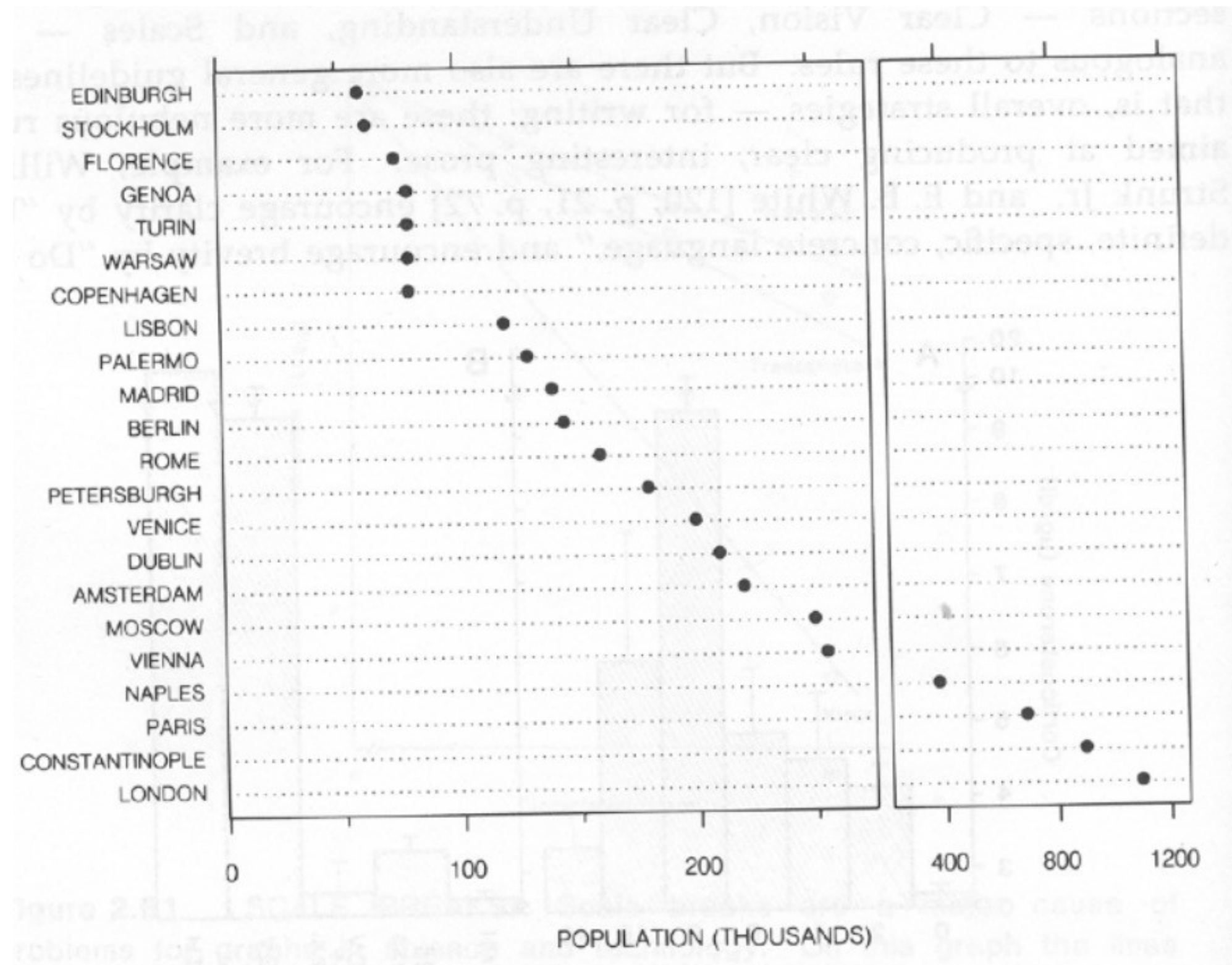




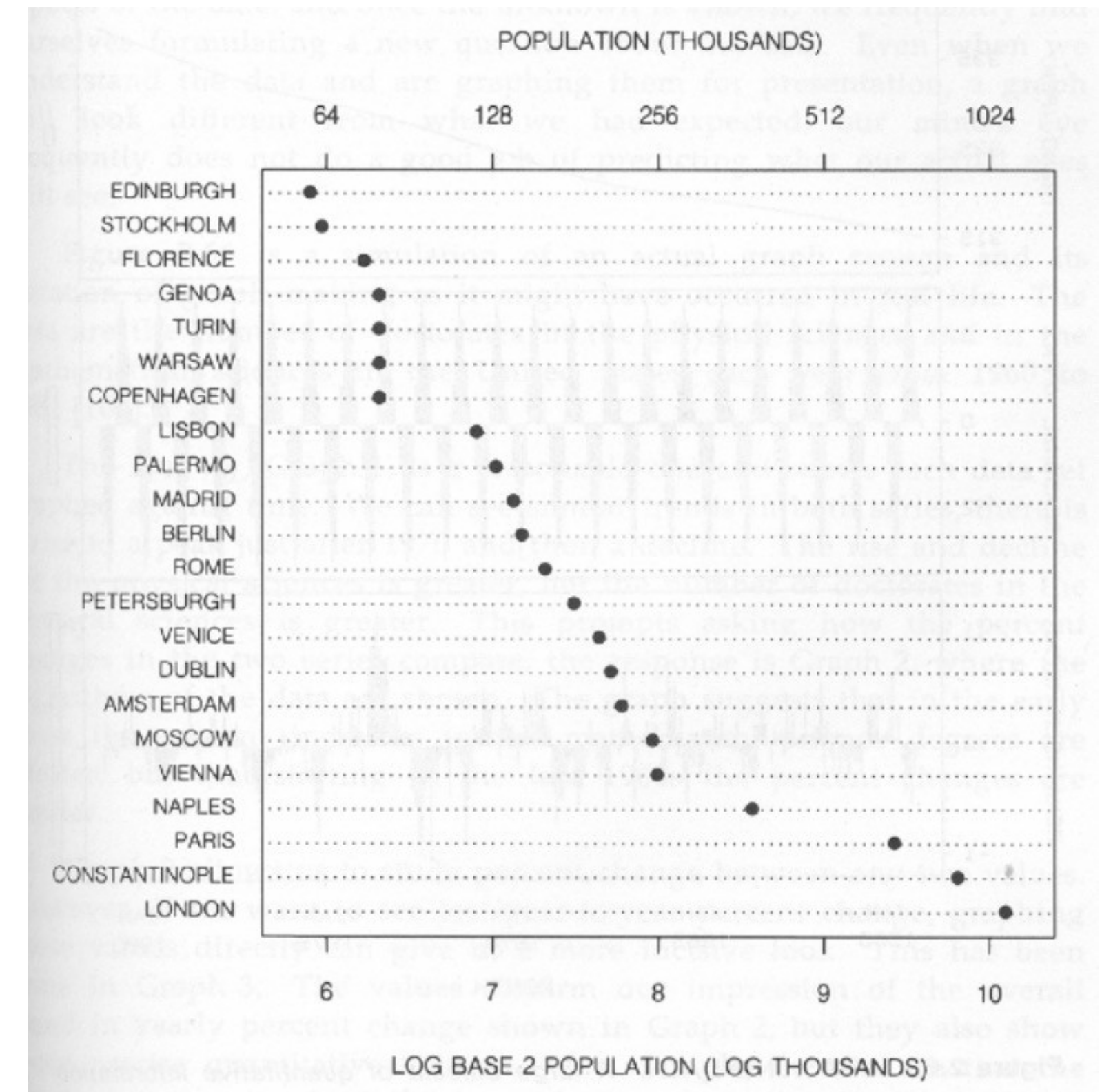
Poor scale break



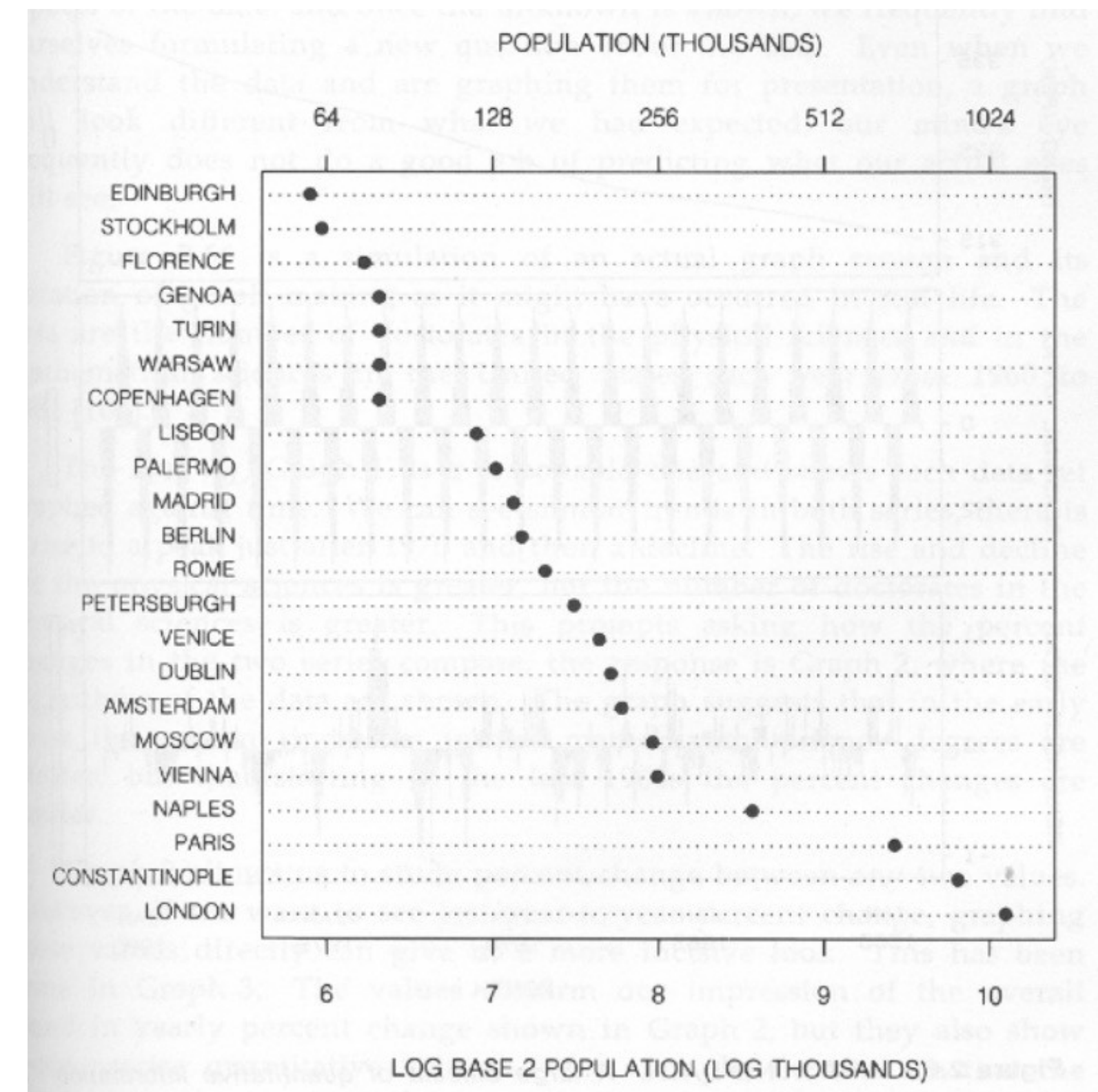
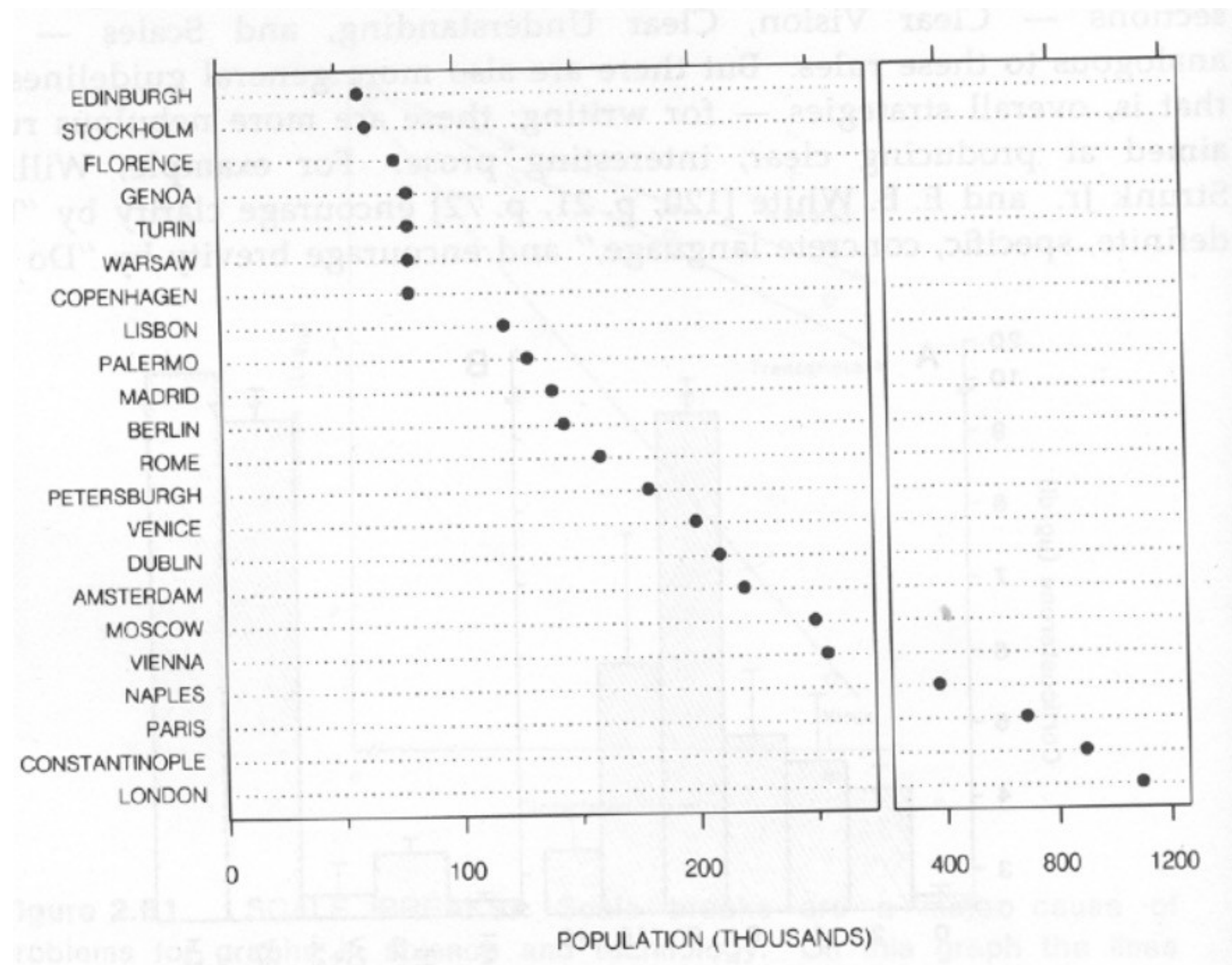
Well-marked scale break



Scale break



Log Scale



- *Scale break: difficult to compare (cognitive - not perceptual - work)*
- *Log scale: direct comparison of all data*

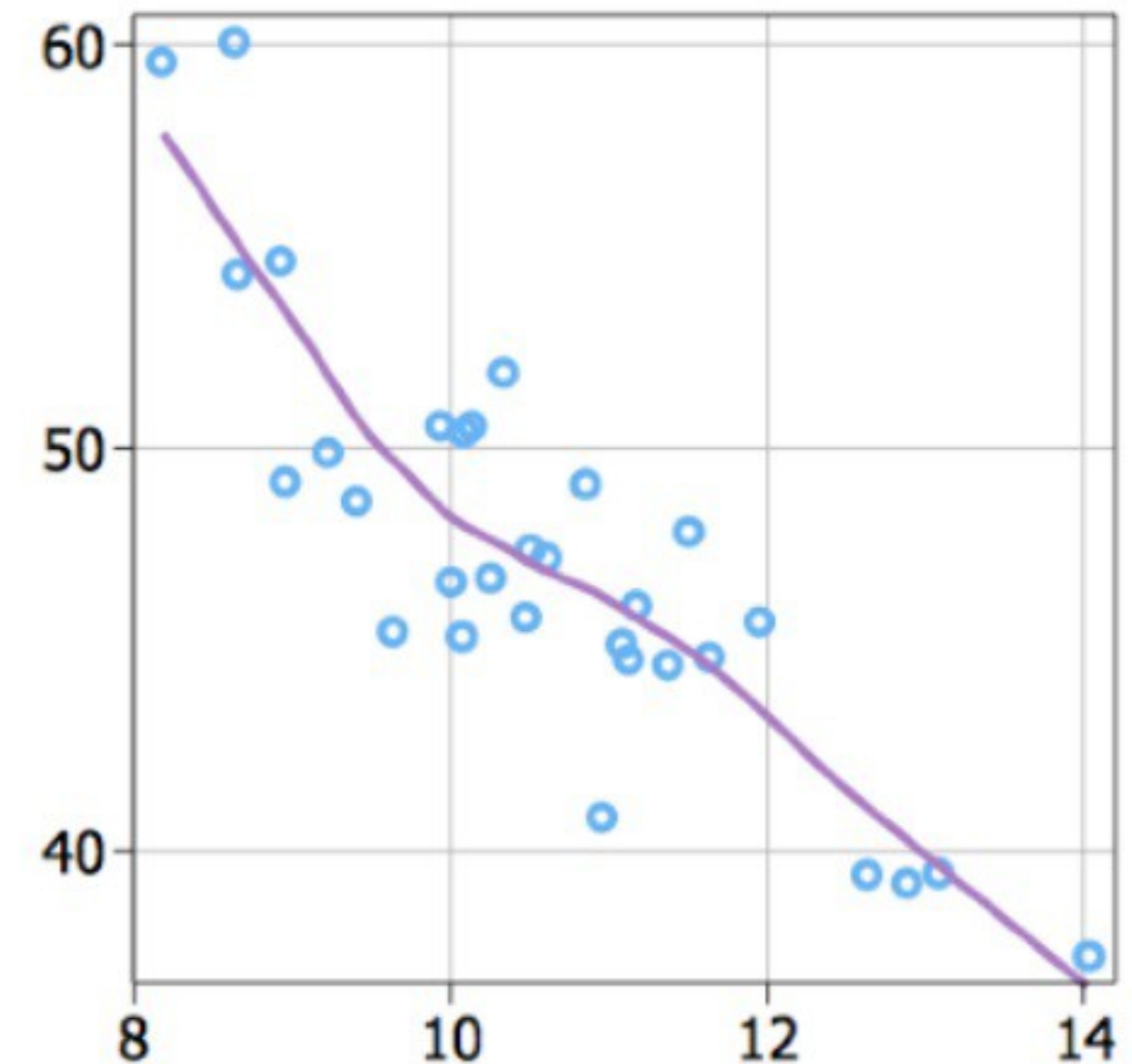
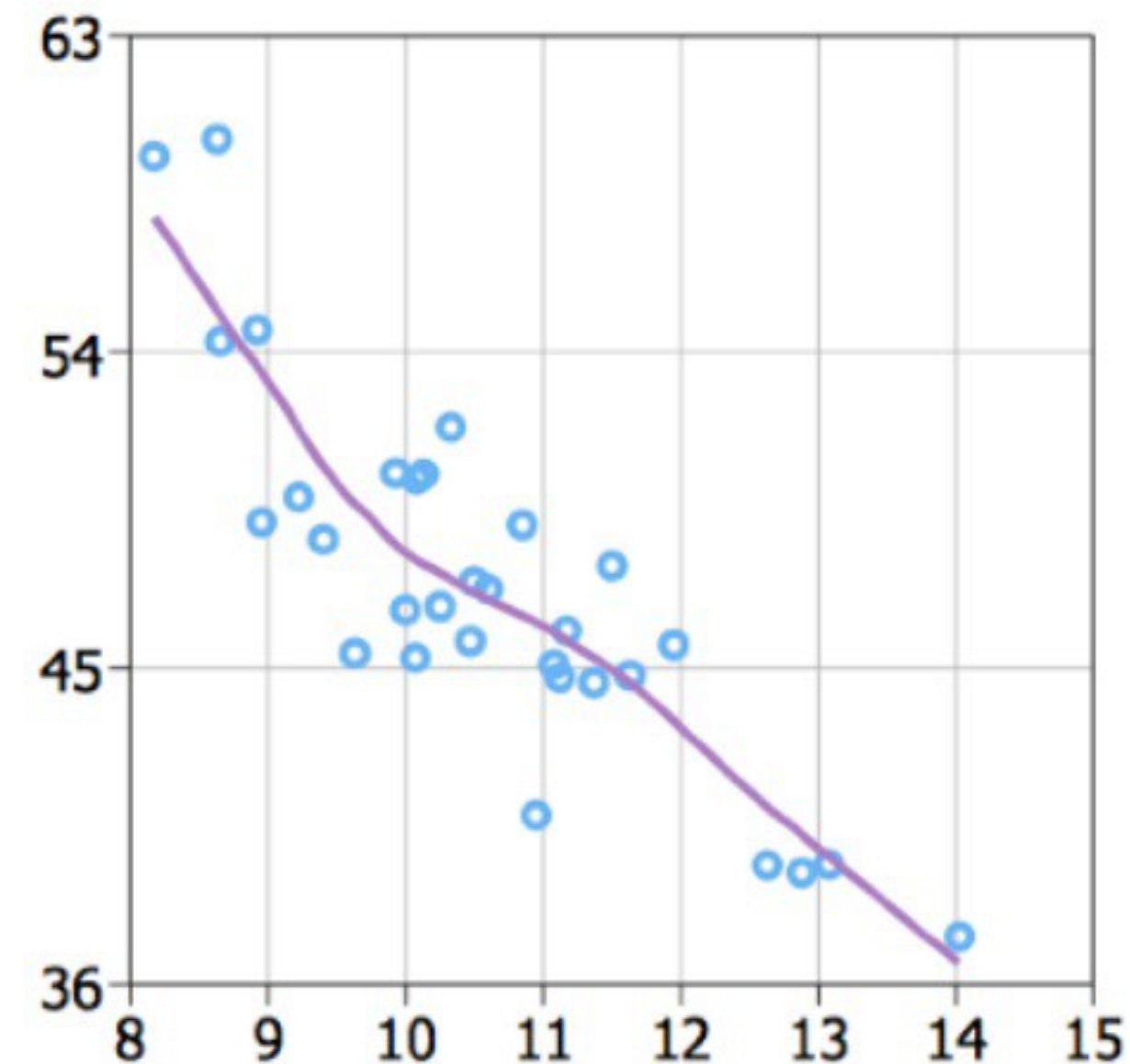
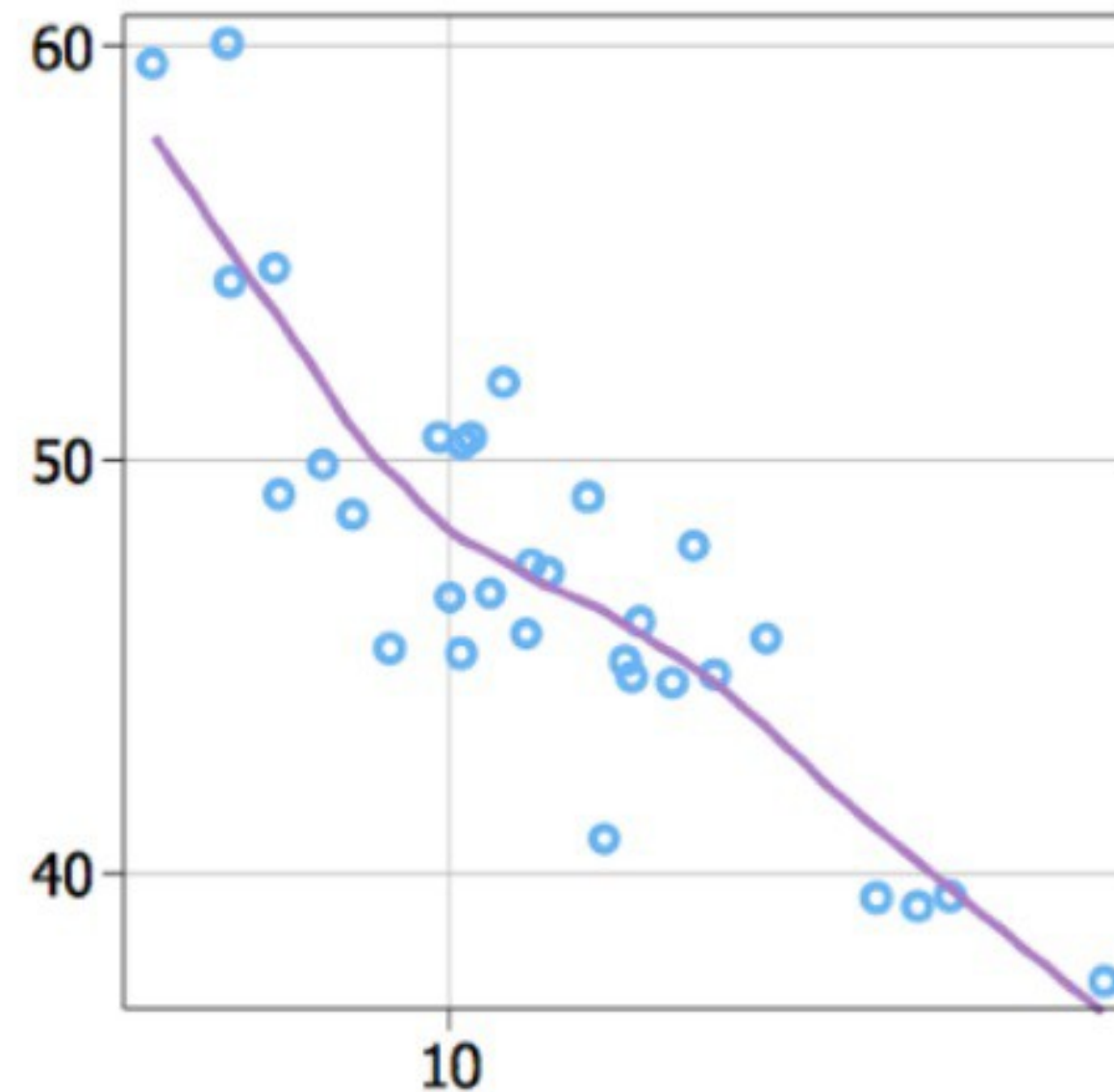
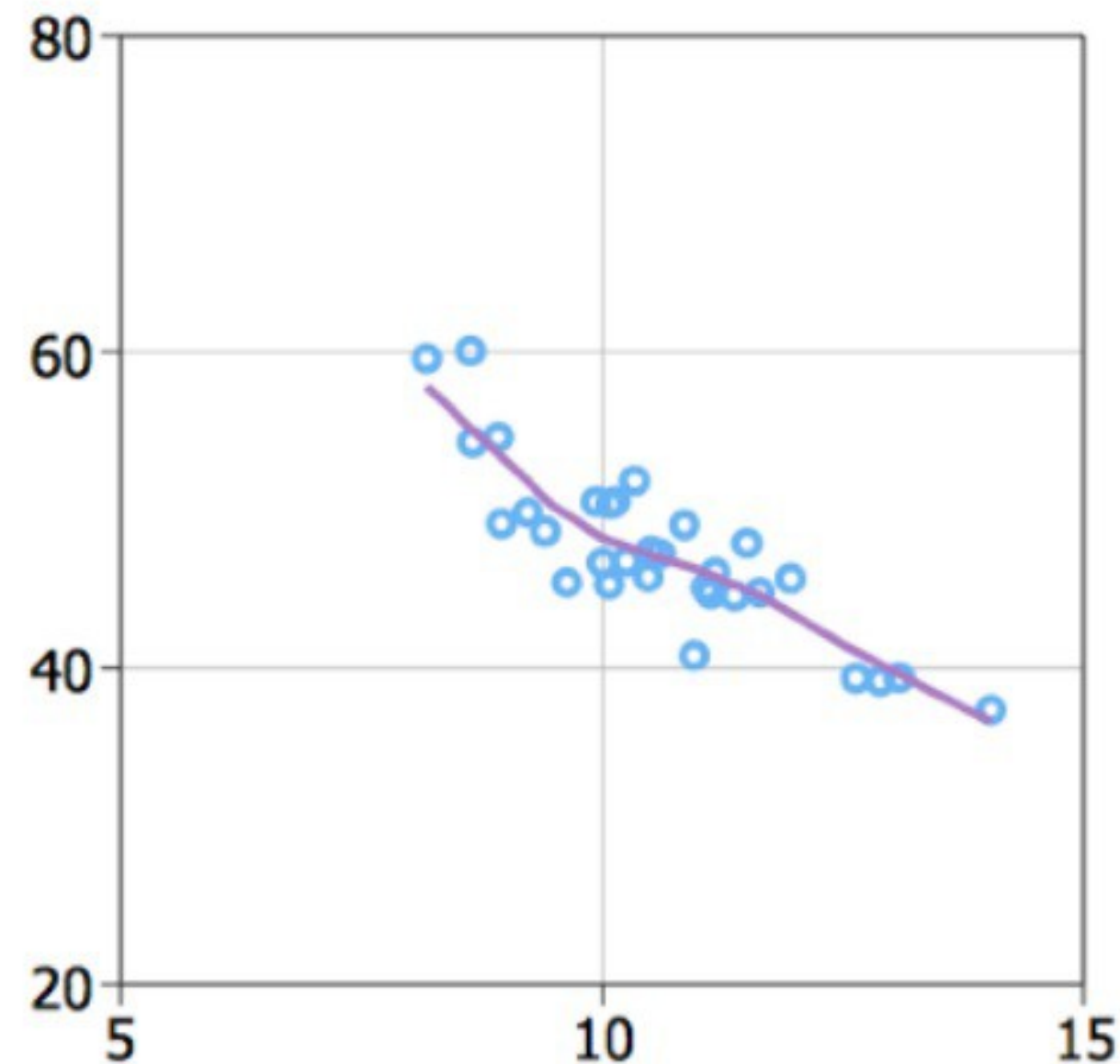
Guidelines: Scales for Position/Length Encodings

Design Considerations

- Linear vs. Log Scale
- Starting from Zero
- Outliers and Scale Break
- Axis Tick Mark

Axis Tick Marks

Guidelines



Simplicity - numbers are multiples of 10, 5, 2

Coverage - ticks near the ends of the data

Density - not too many, nor too few

Legibility - whitespace, horizontal text, size

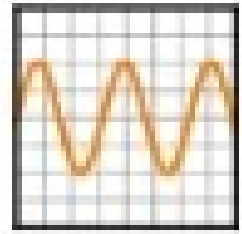
Matplotlib

Matplotlib = "MATLAB Plotting Library"

MATLAB = "Matrix Library"

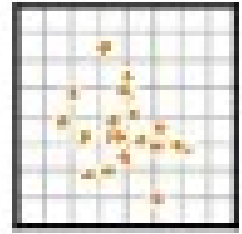
MATLAB is a once popular programming platform for engineers and scientists to analyze data, develop algorithms, and create models.

Basic plots



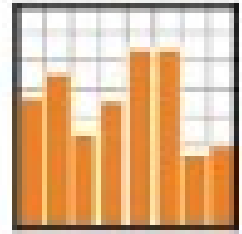
`plot([X], Y, [fmt], ...)`
X, Y, fmt, color, marker, linestyle

API



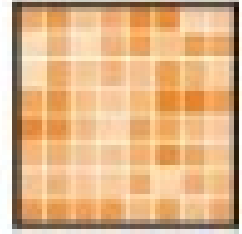
`scatter(X, Y, ...)`
X, Y, [s]izes, [c]olors, marker, cmap

API



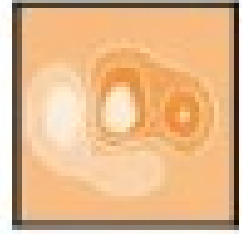
`bar[h](x, height, ...)`
x, height, width, bottom, align, color

API



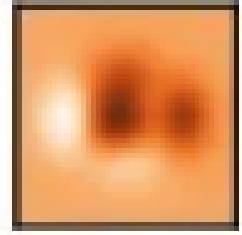
`imshow(Z, ...)`
Z, cmap, interpolation, extent, origin

API



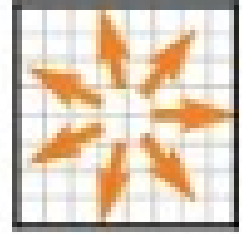
`contour[f]([X], [Y], Z, ...)`
X, Y, Z, levels, colors, extent, origin

API



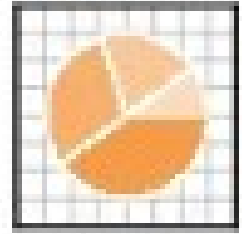
`pcolormesh([X], [Y], Z, ...)`
X, Y, Z, vmin, vmax, cmap

API



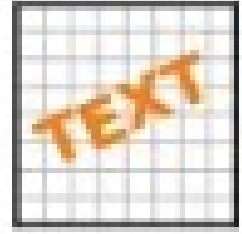
`quiver([X], [Y], U, V, ...)`
X, Y, U, V, C, units, angles

API



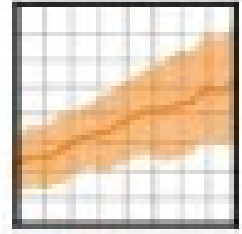
`pie(X, ...)`
Z, explode, labels, colors, radius

API



`text(x, y, text, ...)`
x, y, text, va, ha, size, weight, transform

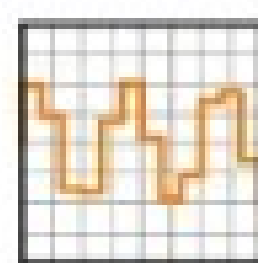
API



`fill[_between][x](...)`
X, Y1, Y2, color, where

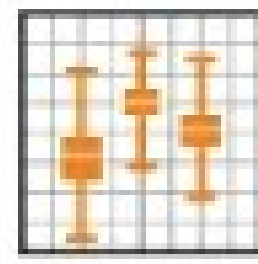
API

Advanced plots



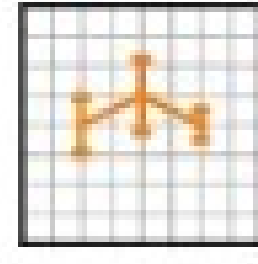
`step(X, Y, [fmt], ...)`
X, Y, fmt, color, marker, where

API



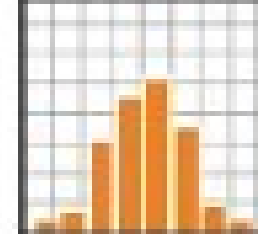
`boxplot(X, ...)`
X, notch, sym, bootstrap, widths

API



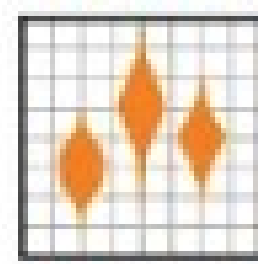
`errorbar(X, Y, xerr, yerr, ...)`
X, Y, xerr, yerr, fmt

API



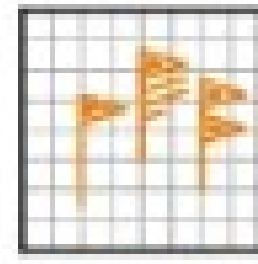
`hist(X, bins, ...)`
X, bins, range, density, weights

API



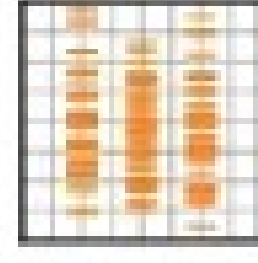
`violinplot(D, ...)`
D, positions, widths, vert

API



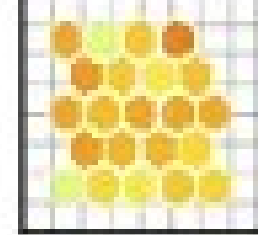
`barbs([X], [Y], U, V, ...)`
X, Y, U, V, C, length, pivot, sizes

API



`eventplot(positions, ...)`
positions, orientation, lineoffsets

API



`hexbin(X, Y, C, ...)`
X, Y, C, gridsize, bins

API

Matplotlib is a **powerful** library that can be used to create **many** kinds of plots

These are a **subset** of the plots you can make

Matplotlib can seem **complicated**

Actually, it **is** complicated

But it is also **powerful** and **consistent** in its **design**

Here are some things to **keep in mind** when learning it

First, everything is based on the ideas
of **Figure** and **Axes**
(and also **plots**)

Basic Structure of a Matplotlib Visualization

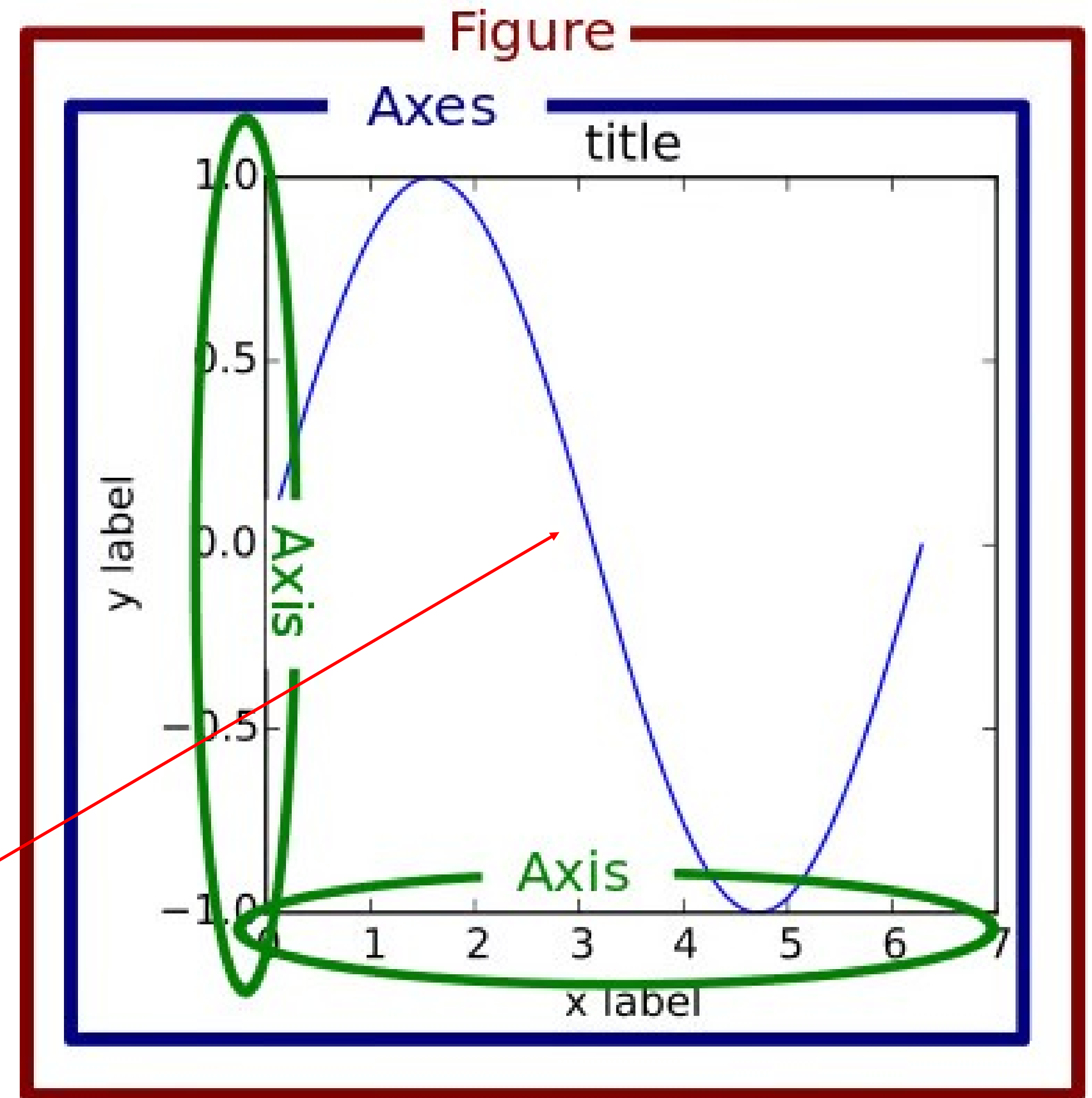
Matplotlib (MPL) has two fundamental concepts:

Figure and Axes

Here is an example plot showing an axes (a pair of axes) **contained** by a figure

The **axes** in this case are the x-axis and the y-axis

A **plotted**
line



Figures

Top level objects that contains **axes**

Think of this as a canvas

Axes

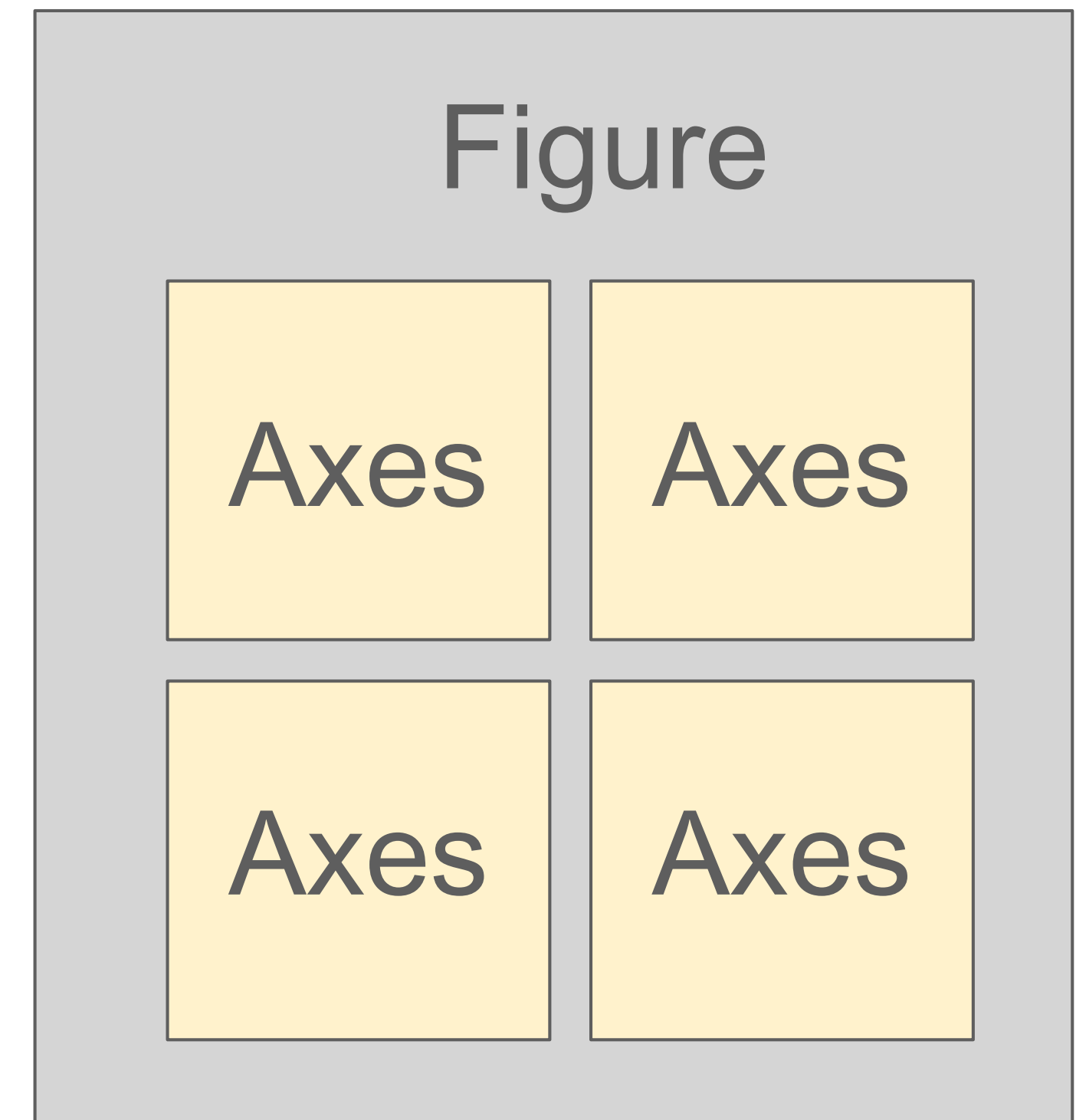
Contain all the elements of an individual **plot** in a figure

Plot / Plotting

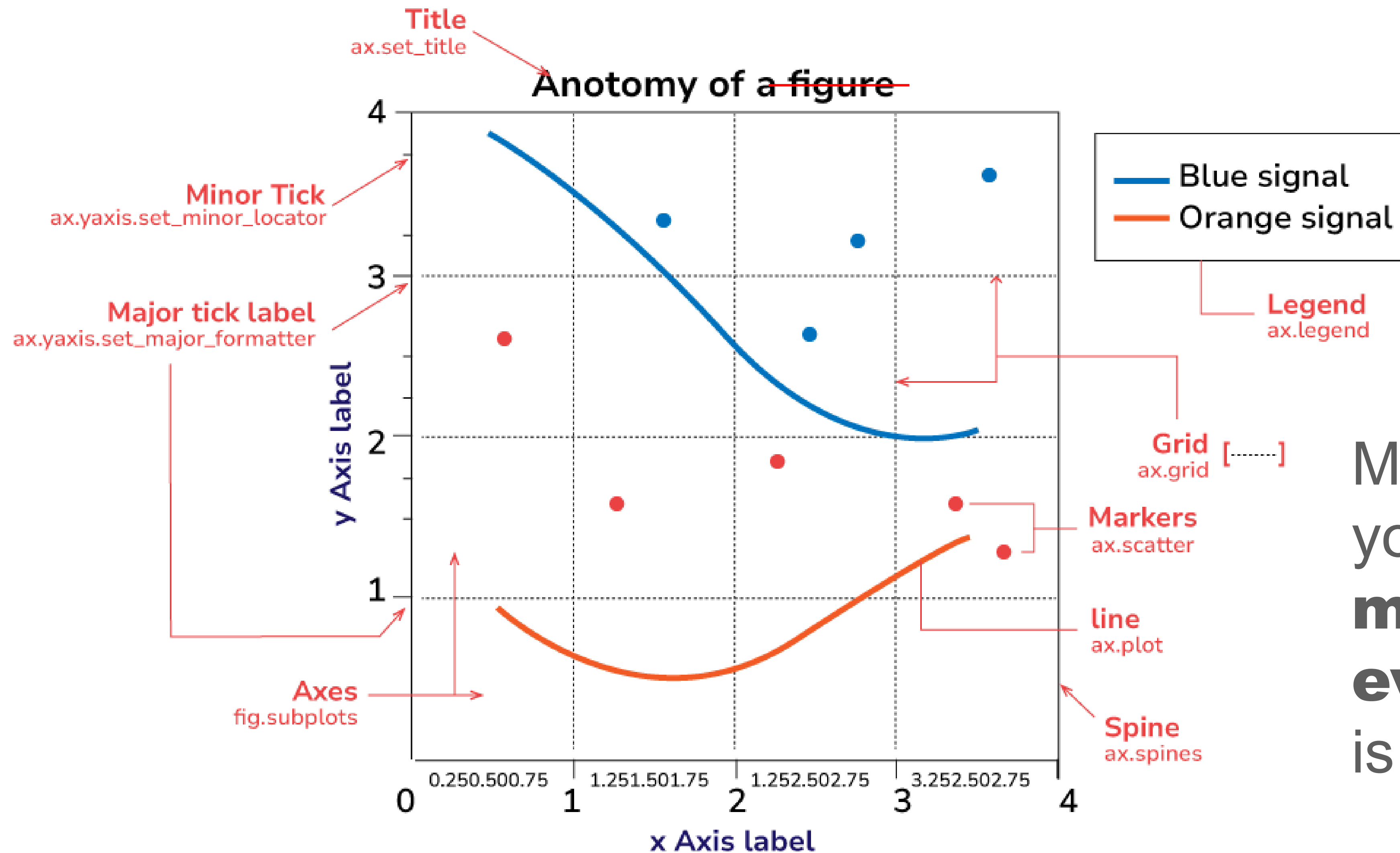
The actual drawings that go on an axes

E.g. Two lines are two plots

Also: the act of plotting



Anatomy of an Axes



Matplotlib enables you to **manipulate everything** that is **named** here

The second thing to keep in mind is the Matplotlib has **two interfaces**
Interactive and **Object Oriented**

In addition, **Pandas** is a wrapper around Matplotlib so it counts as a **third**
This means that example code can be complicated and appear redundant

Sample Code from the Two Interfaces

Operation	Axes interface	pyplot interface
Creating figures	<code>fig, ax = plt.subplots()</code>	<code>plt.subplots()</code>
Plotting data	<code>ax.plot(x, y)</code>	<code>plt.plot(x, y)</code>
Getting properties	<code>label = ax.get_xlabel()</code>	<code>label = plt.xlabel()</code>
Setting properties	<code>ax.set_xlabel("time")</code>	<code>plt.xlabel("time")</code>

Here, **interactive** is called **pyplot**
and **object oriented** is called **Axes**

The best way to **learn** it is by **example**
and by **using it** yourself